UNIVERSITY OF LONDON

CM2020 - AGILE SOFTWARE PROJECTS

FINAL REPORT

Prepared By :

Team 101

Wee Hui Toon
Wong Sui Rong
Wong Wen Li Mandy
Wong Xiang Lin
Devon Maya Xavior

SAGE

Mental Health Companion App

Content Page

1 Introduction

1.1 Background and Objectives

The purpose of this project is to develop a mental health companion app for young adults in Singapore, aiming to fill the gaps left by existing solutions in personalization, empathy and real-time interaction.
For context, there is an increasing prevalence of mental health issues among young adults, particularly depression and anxiety, which have become a significant global and local concern.

The problem that the project addresses is the current inadequacy of other mental health apps in providing personalised and empathetic support. Most apps are good in providing content, but fail to offer real-time, interactive support that the users need during critical moments.
Our project seeks to overcome these limitations by developing an app that integrates advanced AI technology with empathetic design to deliver real-time guidance and personalised support.

The significance of the project is that we can transform the way mental health support is provided to young adults in SIngapore. We hope to create a more supportive and effective environment for managing mental health from our app.

1.2 Project Scope and Aims

For the current scope of our project, the primary deliverables by the final report submission date will include the foundational setup of both the frontend and backend architecture for the three core features: Journal Logging, Mood Tracking, and the Chatbot. As our development progressed, it became evident that additional time beyond the assignment deadline would be necessary to deliver a prototype that meets our standards of quality and functionality.

At this stage, the primary aim is to ensure users can register and log in to their accounts, access the main homepage, interact with our chatbot, and utilise the journaling feature to save and log their entries. Additionally, users will have access to a settings page where they can make basic account modifications (i.e., add display pictures) and delete their accounts if necessary. These core features should operate smoothly.

While future enhancements, such as mental health insights and daily affirmation messages are planned, they fall outside the current scope of this project and will be incorporated in subsequent versions of the website. Further refinement of the existing features will also be addressed in future iterations.

Our ultimate goal is to develop a supportive environment where individuals can effectively manage their mental health and access helpful resources.

2 Research

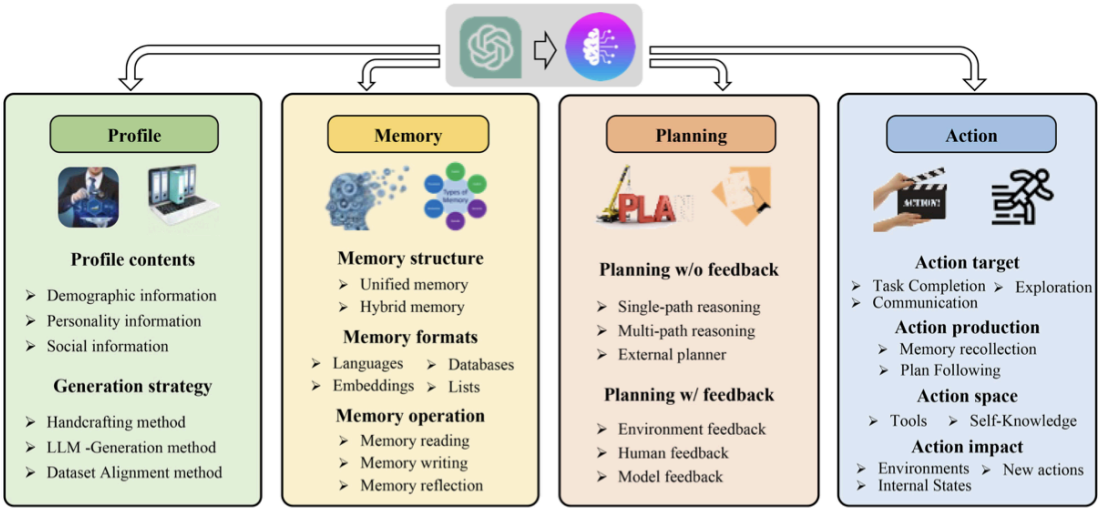
2.1 Review of Relevant Technology

The core features of our mental health companion app—Journal Entry, Mood Tracking, and Chatbot—are meticulously designed to analyse user inputs and deliver responses that are not only intelligent and empathetic but also strikingly human-like. Our objective is to provide actionable advice and robust support, empowering users to manage their mental health more effectively.

To achieve these goals, we have strategically incorporated the **LLM-Powered Autonomous Agent framework** and **AWS Lambda functions** into our development approach. These technologies are instrumental in enhancing the app's capacity to offer personalised and responsive interactions, thereby facilitating a deeper engagement with users and aiding them in their mental health management through consistent and insightful feedback.

In future expansions of the app, we intend to leverage data from the Journal Entry, Mood Tracking, and Chatbot features to conduct comprehensive analyses of users' mental health status. By deriving actionable insights from this data, we will be able to offer users more targeted and relevant resources, including curated articles and helpful links, to further support their mental health journey.

LLM Powered autonomous agent



A unified framework for the architecture design of LLM-based autonomous agent

Souce: (Wang et al., 2024, A survey on large language model based autonomous agents. *Springer Link*, *18*(186345)

An LLM-Powered Autonomous Agent is an advanced system that harnesses the capabilities of **Large Language Models (LLMs)** to autonomously perform complex tasks, emulating human-like comprehension and decision-making processes. These agents leverage the sophisticated natural language processing features of LLMs to understand user inputs, generate contextually relevant responses, and execute tasks independently, minimising the need for continuous human oversight. (Weng, 2023)

By integrating extensive datasets and advanced algorithms, LLM-Powered Autonomous Agents are capable of learning, adapting, and responding intelligently to a diverse array of scenarios. (Wang et al., 2024) This makes them particularly effective in applications that demand nuanced communication, such as virtual assistants, chatbots, and interactive support systems. (Weng, 2023)

In the context of our mental health companion app, these agents are instrumental in delivering empathetic, personalised, and insightful interactions, which are crucial for supporting users in their mental health management. To achieve this, we have selected **OpenAI's GPT-3.5 turbo/4.0 mini** as the foundational LLM for our chatbot feature and have structured our development around the LLM-Powered Autonomous Agent framework. This approach ensures that our app not only meets user needs but also evolves in response to the complexities of mental health care. (Refer to section 5.3 to see how the framework is applied to our mental health companion chatbot)

AWS Lambda function

AWS Lambda is a serverless computing service that allows developers to run code in response to specific events without provisioning or managing servers. This service automatically scales to accommodate the workload, ensuring that tasks are executed efficiently and only when needed. (Mishra, 2024)

Lambda functions can be triggered by various events, such as HTTP requests, database updates, or file uploads, making them versatile for handling a wide range of use cases. In a development environment, Lambda functions help to simplify architecture by offloading specific tasks from the main application, leading to enhanced performance, reduced operational costs, and increased agility.

The ways we hope to leverage on the Lambda function for our app is in data processing, notification triggers, integration with external APIs (i.e., OpenAI's GPT models), generation of automated reports and analytics, and also management of specific user requests.

Area	How it applies to our app
Data Processing	When a user submits a journal entry/updates their mood, a Lambda function can be triggered to analyse the data for patterns or keywords. This approach ensures that data processing is offloaded from the main server, keeping the app responsive while minimising resource consumption.
Notification Triggers	If a user's mood analysis warrants an alert, a Lambda function can automatically send notifications without the need for a dedicated service running continuously. This ensures timely communication with the users.
Integration with external APIs	When the app needs to generate mood insights or chat responses based on user interactions, a Lambda function can handle the API call, keeping the main backend code clean and ensuring resources are used only when necessary. This modular approach simplifies the architecture by isolating external interactions to specific functions.
Automated reports/analytics	By processing user data at regular intervals or on demand, the lambda functions enable scalable and efficient data handling without the need for continuous server operation. This is especially useful when the user data grows and large volumes of data have to be processed.
Management of user requests	Each user request is processed independently, allowing for multiple requests to be handled simultaneously, enhancing user experience.

We chose to work with lambda functions because it offers cost-efficiency by executing tasks only when needed, scalability by adjusting to demand automatically, and simplification by offloading specific tasks from the main server. For e.g., Lambda will offload event-driven operations from ECS, allowing the latter to focus on core services that require a persistent state, like user session management and continuous mood monitoring. Ultimately, using lambda in conjunction with ECS will help our team to deliver a responsive, scalable and user-centric application.

*Please note that at the point of report submission, our app remains in its foundational stage, and thus, AWS Lambda functions have not yet been integrated. Our primary focus is on stabilising the basic code architecture before incorporating additional enhancements. However, moving forward, we fully intend to integrate Lambda functions as a key component of the app's development to optimise performance and scalability.

3 Planning and Methodology

3.1 Project Planning

The planning of our mental health companion app was initially outlined in the midterm assignment, leveraging the **Software Development Lifecycle (SDLC) framework.** This provided a structured approach for defining the primary tasks, which were subsequently organised and managed using the **Agile Scrum framework**, complete with detailed milestones and timelines (refer to Appendix A.3).

However, as this project marks our team's first experience with the Agile Scrum methodology, we encountered challenges in adhering strictly to the planned milestones and timelines. Specifically, we faced delays in several coding tasks due to the additional time required for debugging and refining the code. Consequently, adjustments to the initial plan were necessary, with several deadlines extended to accommodate these changes (based on the contingency measures drawn out in the midterm report). These adjustments and their impact on the project will be thoroughly discussed in subsequent sections of this report.

3.2 Resource Allocation and Management

	Member	Tasks/Skills
Frontend	**Mandy Xiang Lin**	**Figma, Javascript, REACT Stack, TailwindCss, UX/UI design, Github, AWS services**
Backend	**Sui Rong Hui Toon Devon**	**Figma, Node.js Stack, SQlite, OpenAI GPT API, Postman API testing, Github, AWS services**

In our Agile software project for the mental health companion app, our team has clearly defined roles and responsibilities to ensure efficient collaboration and progress.

At the **frontend**, Mandy and Xiang Lin are focused on the frontend development, where they design and implement user interfaces that are both aesthetically pleasing and user-friendly. Their work involves creating responsive layouts, integrating design elements, and ensuring that the app provides a seamless user experience.

On the **backend**, Sui Rong, Hui Toon and Devon are responsible for developing the server-side logic, managing databases, and integrating APIs, including the OpenAI API for the chatbot functionality. They work closely with the frontend team to ensure that the data flows smoothly between the client and server, resulting in a cohesive and robust application. This collaboration under the Agile framework allows us to adapt quickly to changes, continuously improve the app, and meet the evolving needs of our users.

Each sub-team (i.e., frontend or backend) worked on their own tasks but met up as a whole project group every 1 or 2 weeks to update on the overall development of the app. Additionally, the team has set up a repository on **Github** to manage version control for both frontend and backend teams. Last but not least, meeting notes and resources were stored in a shared document on **Google Drive** for everyone's access and documentation..

3.3 Timeline and Milestones

Frontend

The frontend team began by conducting thorough research on the appropriate tech stack for developing the web app's frontend, dedicating approximately 1.5 weeks to this process. Following the research phase, the team carried out pre-surveys and interviews to gather comprehensive user stories and insights, which served as the foundation for designing the app's wireframes. Once these preliminary tasks were completed, the team established the project's repository on GitHub and created the necessary documentation files. This preparation allowed the team to move forward with developing the code skeleton and subsequent tasks efficiently.

Tasks	Start Date	Completed	Backlog	Member
Setup SCRUM framework & research on React stack	1st July 2024	Tasks and Sprints planning done	Wireframes	All
Pre-survey and Interviews on user stories and needs	3rd July 2024	Collected survey and interview data	Data analysis	All
Working on Midterm Report (3rd - 8th July 2024) / Other assignments (9th - 22nd July 2024)				
Set up Github repository and Google shared folder	23rd July 2024	Repository "Team101" set up, google doc created with meeting notes recorded	Code Skeleton	Mandy (Github) Xiang Lin (Google Drive)
Select tech stack and initiate frontend directory: Installing libraries and dependencies (i.e., install node.js, react, tailwind css, axios, vite)	24th July 2024	Frontend code skeleton	Building of main pages (i.e., landing page, homepage etc)	Mandy Xiang Lin
Landing page, Register page, Login page	28th July 2024	Finished setting up register and login pages functions with links to homepage once logged in	Data preload Link to database (yet to test with dummy data)	Mandy Xiang Lin
Navigation bar	4th August 2024	Set up respective routes to link to other pages		Mandy Xiang Lin
Homepage	5th August 2024	Set up real-time date/time Daily journal input box done Journal save entry button functionality done Journal log link button functionality done Daily recommended insights and articles functionality done	Greeting message not linked to user account Weekly calendar needs debugging Daily affirmation quote needs debugging Daily recommended insights and articles needs linking to API	Mandy Xiang Lin
Chatbot	12th August 2024	Chatbot functionality done	Need to link to API - GPT - 3.5 turbo or 4.0 mini Need to link to database	Devon
Mood Tracker	13th August 2024	Mood bar done Mood input box and	Need to link API Need to link to	Devon

		save functionality done Mood history functionality done	database	
Insights	14th August 2024	Summary box done Mood distribution box done Resources link box done	Need to link to API Need to link to database	Devon
Profile page	16th August	Display image functionality done	Profile save functionality Feedback functionality Account deletion functionality Profile page not linked to database	Mandy
README doc with instructions	18th August 2024	Instructions for starting up the app	Configure the code such that only logged in users can access /homepage	Mandy
Syncing up with backend	19th August 2024	Synced up database to account registration/login	Bug with SQlite port	Mandy Sui Rong
Documentation and Report	20th August 2024	Document project progress and report writing		All

Backend

The backend team began by acquiring knowledge in building APIs and familiarising themselves with testing platforms like Postman, essential for the future validation and debugging of their code. To ensure the chatbot feature would meet the goal of being an empathetic, human-like companion, we explored and tested OpenAI's GPT-3.5 turbo and 4.0 mini models on the OpenAI playground. Preliminary user testing indicated that the 4.0 mini model performed better in generating empathetic responses (Starks, 2024). However, given that our app is still in its foundational development stage, we acknowledge that we have not yet fully explored the 4.0 mini model's capabilities. Once we have conducted more comprehensive research and testing, we anticipate transitioning to this more advanced model to optimise the chatbot's functionality.

Tasks	Start Date	Completed	Backlog	Member
Setup SCRUM framework & research on React stack	1st July 2024	Tasks and Sprints planning done	Wireframes	All
Pre-survey and Interviews on user stories and needs	3rd July 2024	Collected survey and interview data	Data analysis	All
Working on Midterm Report (3rd - 8th July 2024) / Other assignments (9th - 22nd July 2024)				
Set up Github repository and Google shared folder	23rd July 2024	Repository "Team101" set up, google doc created with meeting notes recorded	Code Skeleton	Mandy (Github) Xiang Lin (Google Drive)
Identify backend functionalities (setting up team account etc)	24th July 2024	User authentication Database mgmt API endpoints Integration with external services (i.e., GPT 3.5 turbo) Explored Postman API testing platform	OpenAI API key	Hui Toon Sui Rong Devon
Select tech stack and initiate backend directory: Installing libraries and dependencies (i.e., install node.js, express.js, react, tailwind css,SQlite, body-parser, cors, dotenv, openai, sequelize)	26th July 2024	Backend code skeleton	AWS services	Hui Toon Sui Rong Devon
Configure server and framework	31st July 2024	Server file .env file (database credentials, API keys)	Database	Hui Toon Sui Rong Devon
Database	8th August 2024	SQlite database Create sequelize model for users Seed data for testing	Register function Log in function	Hui Toon Sui Rong Devon
User authentication Error handling	16th August 2024	Implement user authentication Codes for handling error	API integration	Hui Toon Sui Rong Devon
User authentication testing	18th August 2024	Successful testing with Postman	API integration	Sui Rong
Chatbot	20th August 2024	Chatbot functionality	Lambda functions	Sui Rong

		Integration of GPT 3.5-turbo		
Syncing up with Frontend	23rd August 2024	Chatbot functionality works	SQliteL port inaccessible	Sui Rong
Documentation and Report	25th August 2024	Document project progress and report writing		All

3.4 Planning Adjustments

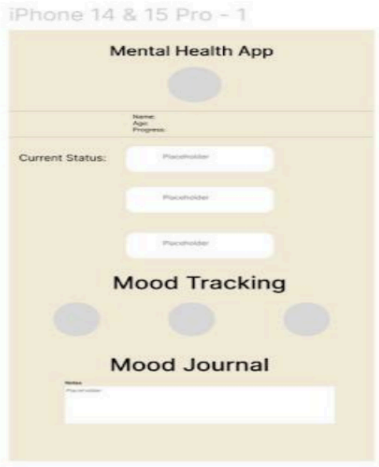
We faced a few challenges managing team availability and balancing workloads. To address these, regular team meetings were held to review progress and reallocate tasks as needed, ensuring that any bottlenecks were promptly addressed. Due to time constraints caused by the extra time needed for debugging, the scope of the features for the final project submission was refined to focus on core functionalities (i.e., Journal entry, Mood tracking and Chatbot). Additional features like Goal setting/Progress tracking, Guided meditations/useful articles, secure therapist sessions and community forum have been rescheduled to be released at the projected date in November 2024.

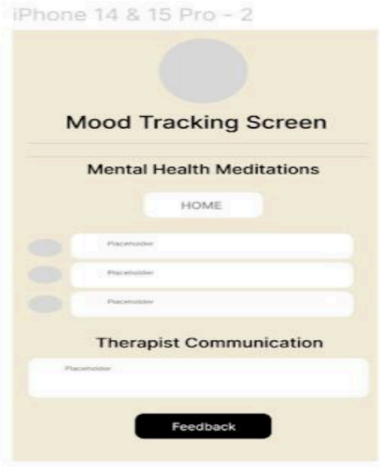
4 Prototyping and Iteration

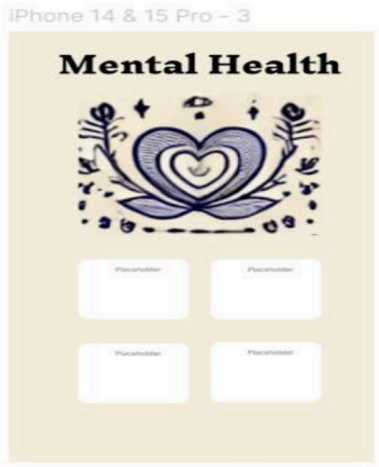
4.1 Initial Prototypes

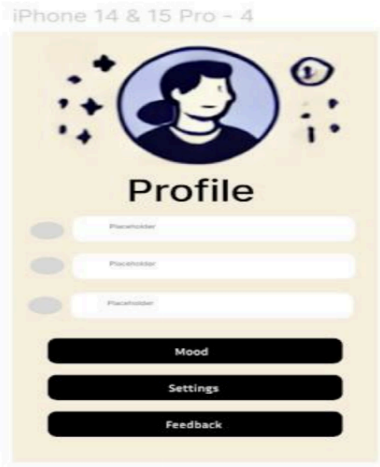
At the outset of the project, as detailed in the midterm report, we developed initial prototypes in the form of mobile web wireframes **(see images below)**. These wireframes were designed based on insights from pre-surveys and interviews, which highlighted user preferences for a *"clear, simple, and minimalistic"* interface with intuitive navigation and minimal complexity **(refer to Appendix A.6)**.

However, subsequent user testing of these wireframes revealed several areas for improvement. Users reported that the colour scheme was unappealing and that the user interface lacked sufficient detail, leading to a less engaging experience. Additionally, the navigation was not sufficiently streamlined, which contributed to a sense of disconnection and confusion among users. This feedback underscored the need for iterative refinements to enhance visual appeal and usability, ensuring a more cohesive and user-friendly design.





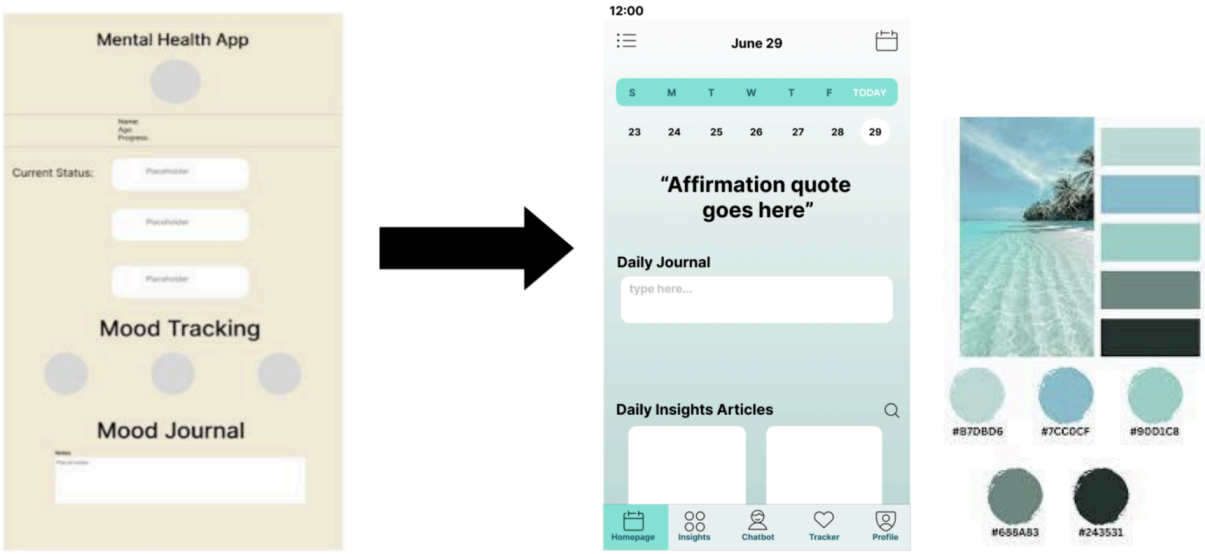




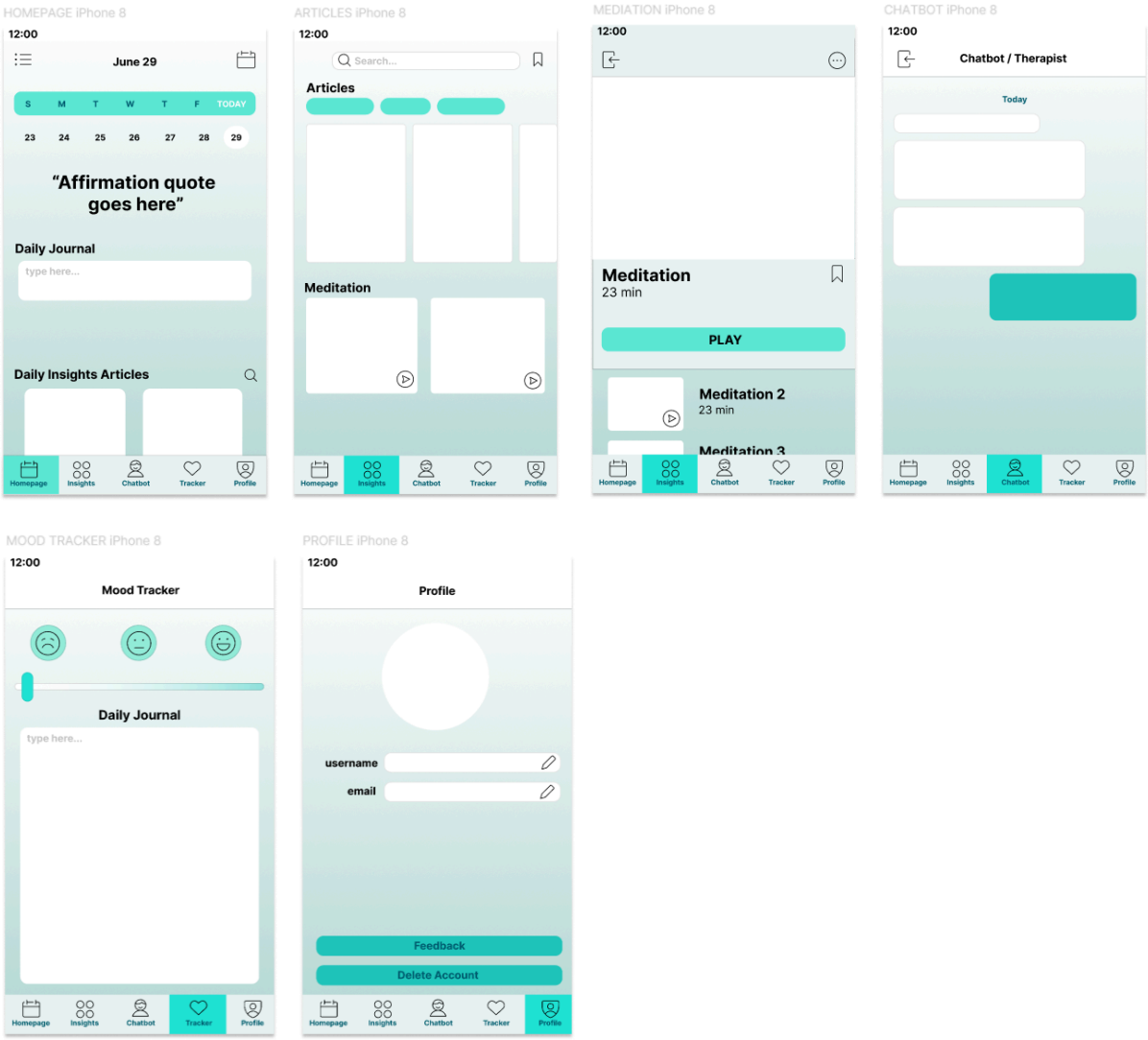
4.2 Design Process

In the second iteration of our design process, we focused on enhancing the aesthetic appeal and functionality of our initial interface, guided by the principles of intuitive design. We meticulously refined the colour story and layout to better align with user expectations and improve overall usability. Through an analysis of competitive apps in the market, we identified common design patterns that users are familiar with, such as the placement of the hamburger menu icon in the top-left corner, a weekly calendar prominently displayed at the top, a large-font affirmation quote to capture attention, a spacious text-input box for daily journal entries, scrollable sections for daily insights and articles, and a streamlined navigation bar at the bottom for quick access to frequently used features, aligning with the thumbs rule design.

Given the app's focus on mental health, we also aimed to create an emotionally resonant experience. To achieve this, we selected a colour palette of cool and calming teal shades, designed to foster a sense of relaxation and tranquillity within the interface. This approach not only enhances the visual appeal but also supports the app's therapeutic goals by creating a soothing user environment.



Low-fidelity Prototype (Stage 2)



4.3 Design Decisions

Feasibility Study

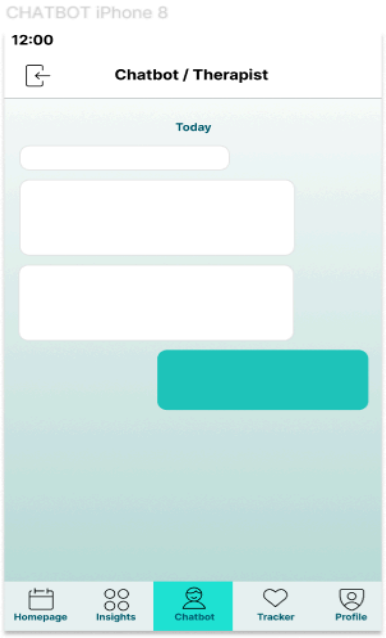
At this stage of our design iteration, our team recognized the necessity of conducting a feasibility study to assess whether we could deliver a mobile app of the desired quality within the constraints of our release timeline. The following analysis outlines our findings:

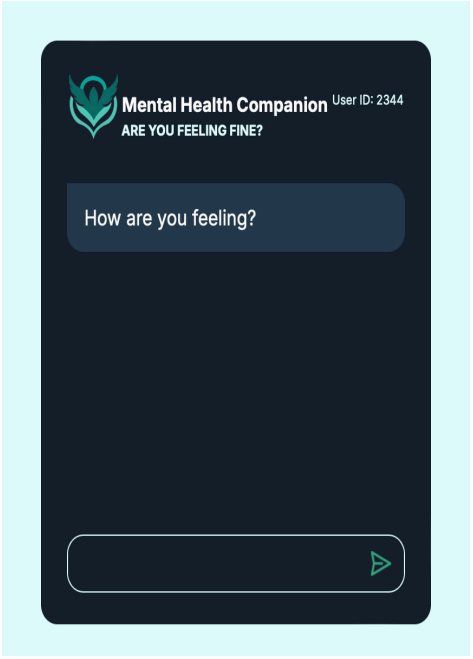
Technical	Our mental health companion app is technically feasible given the current resources and technology. The app will be built using web technologies such as React for the frontend and Node.js for the backend. These technologies are well within our team's expertise, and the necessary tools and libraries are readily available. However, developing a mobile app version is currently beyond our technical capabilities due to the lack of experience in mobile app development frameworks like React Native or Flutter. Moreover, the additional complexity and time required for mobile app development exceed our current capacity.
Economic	From an economic standpoint, at the prototype stage, developing a web app is more cost-effective. The web app can be accessed across multiple devices, including smartphones, tablets, and desktops, without the need for separate development tracks. This approach reduces development costs and allows us to allocate resources more efficiently. Moreover, the hosting and maintenance costs for a web app are lower compared to those for a mobile app, which requires distribution through app stores and ongoing updates for multiple platforms.
Schedule	Given the current project timeline, developing a web app is the most feasible option. The web app can be developed and deployed within the scheduled time frame, allowing us to meet our project deadlines. Developing a mobile app would require extending the timeline significantly, which is not feasible given our current commitments and resource constraints. By focusing on the web app, we can ensure that the core features are developed and delivered on time, while still leaving the possibility open to expand to mobile platforms in the future.

Based on the outcomes of our feasibility study, our team concluded that developing a mobile app to the high standards we envisioned would not be feasible within the given time and resource constraints. Consequently, we have decided to prioritise the development of a web app, ensuring it is accessible across various devices, including mobile phones, tablets, and desktops. This approach allows us to maintain the quality and functionality necessary for our mental health companion app while adhering to our project timeline.

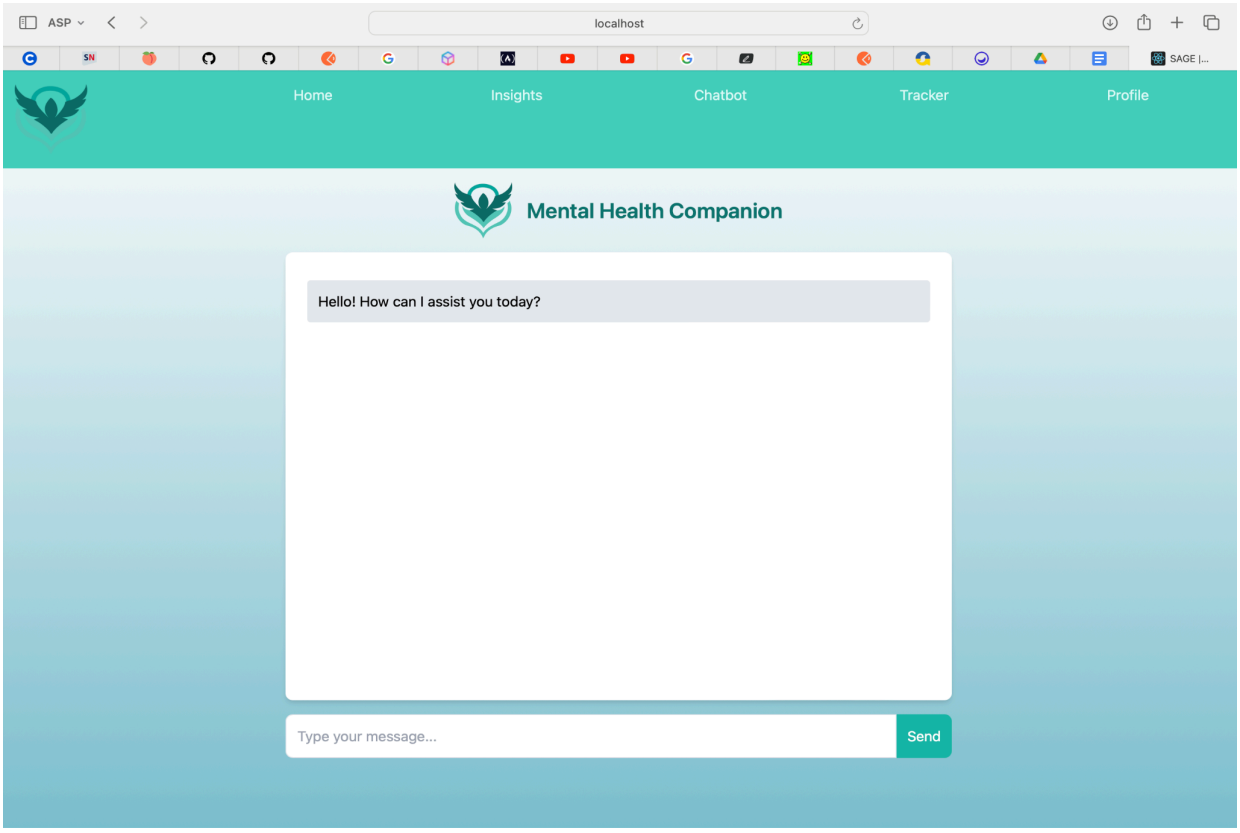
Design Transition

Wireframe





Mobile App



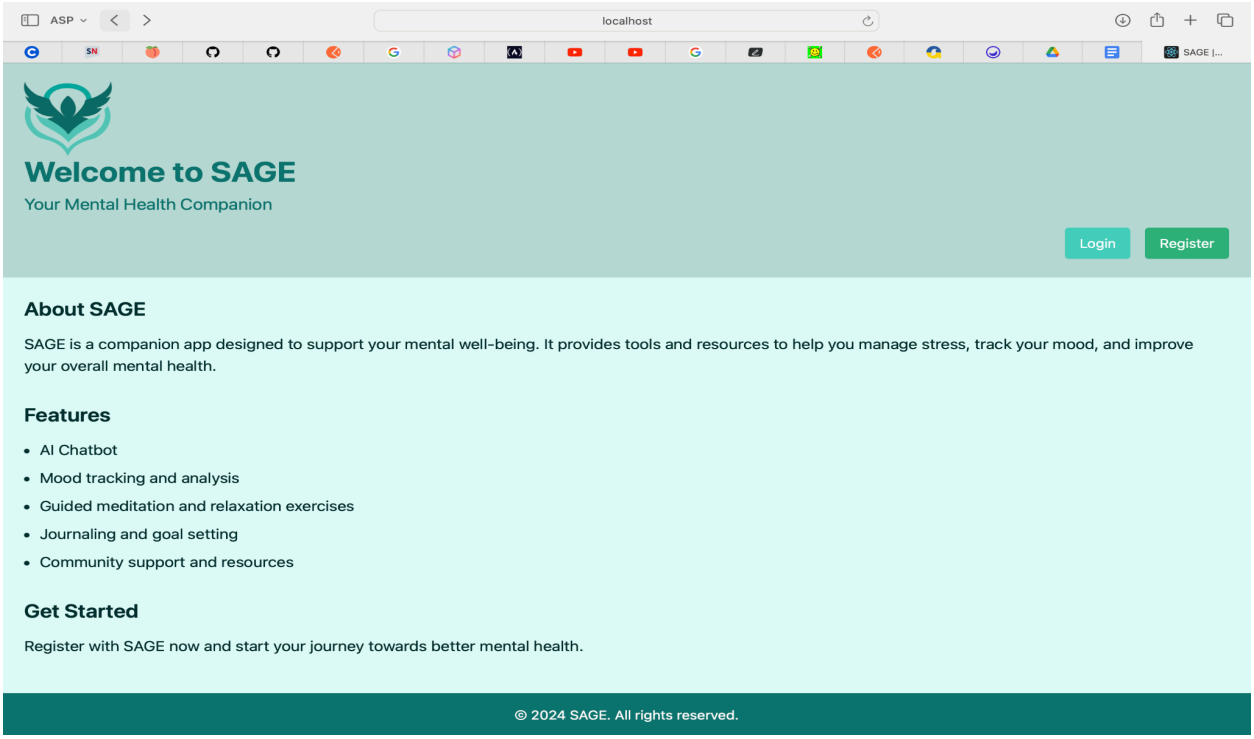
Web App

5 Design Specifications

5.1 Final Design Overview

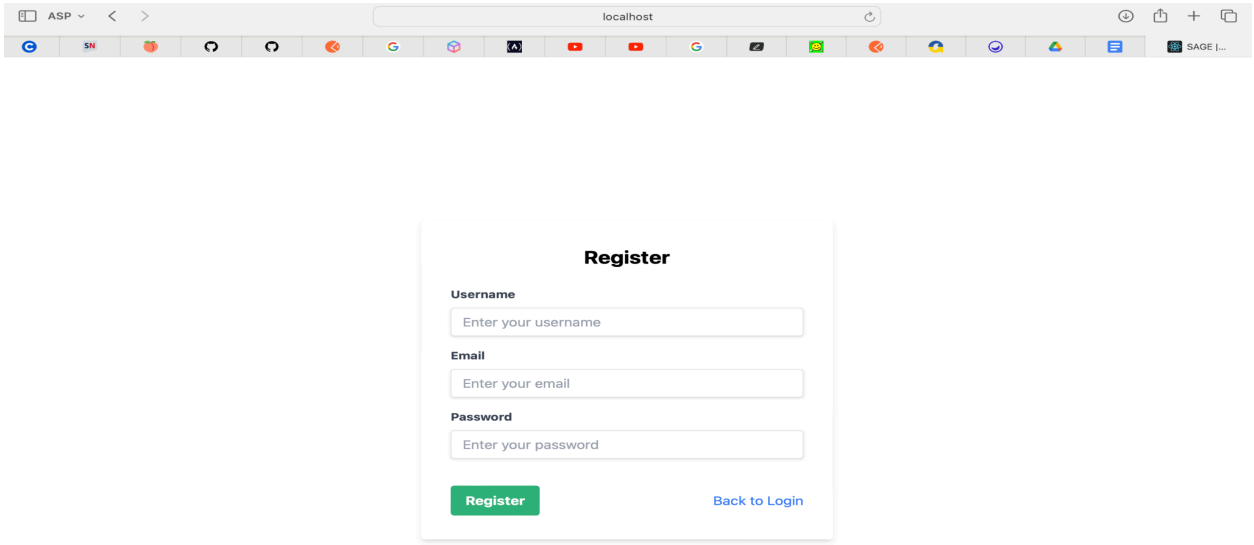
Our final design is a comprehensive mental health companion web app, a platform that has been developed with a user-centric approach, ensuring that it provides seamless functionality and an intuitive interface, regardless of the device used.

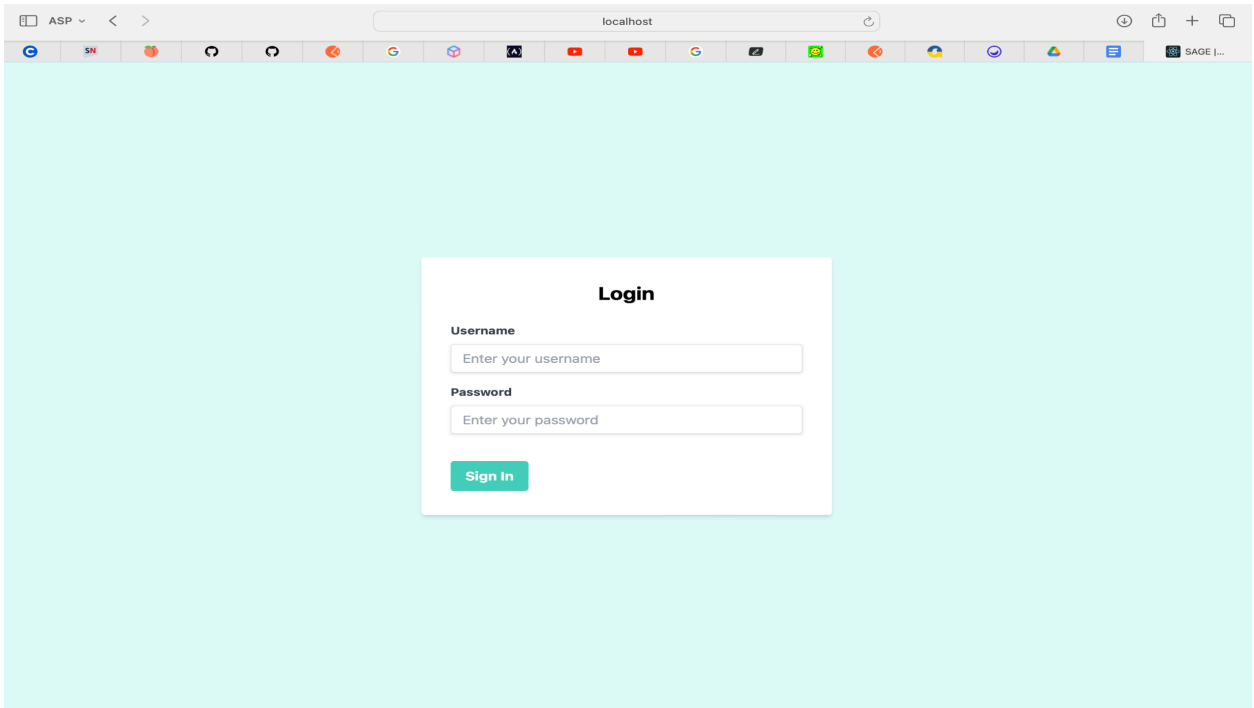
Landing page



When users first access our mental health companion web app, they are welcomed by a thoughtfully designed landing page. This page provides a clear introduction to the app's purpose and guides users on how to begin their journey. From there, they can either create a new account or log in with existing credentials to start exploring the app's functionalities.

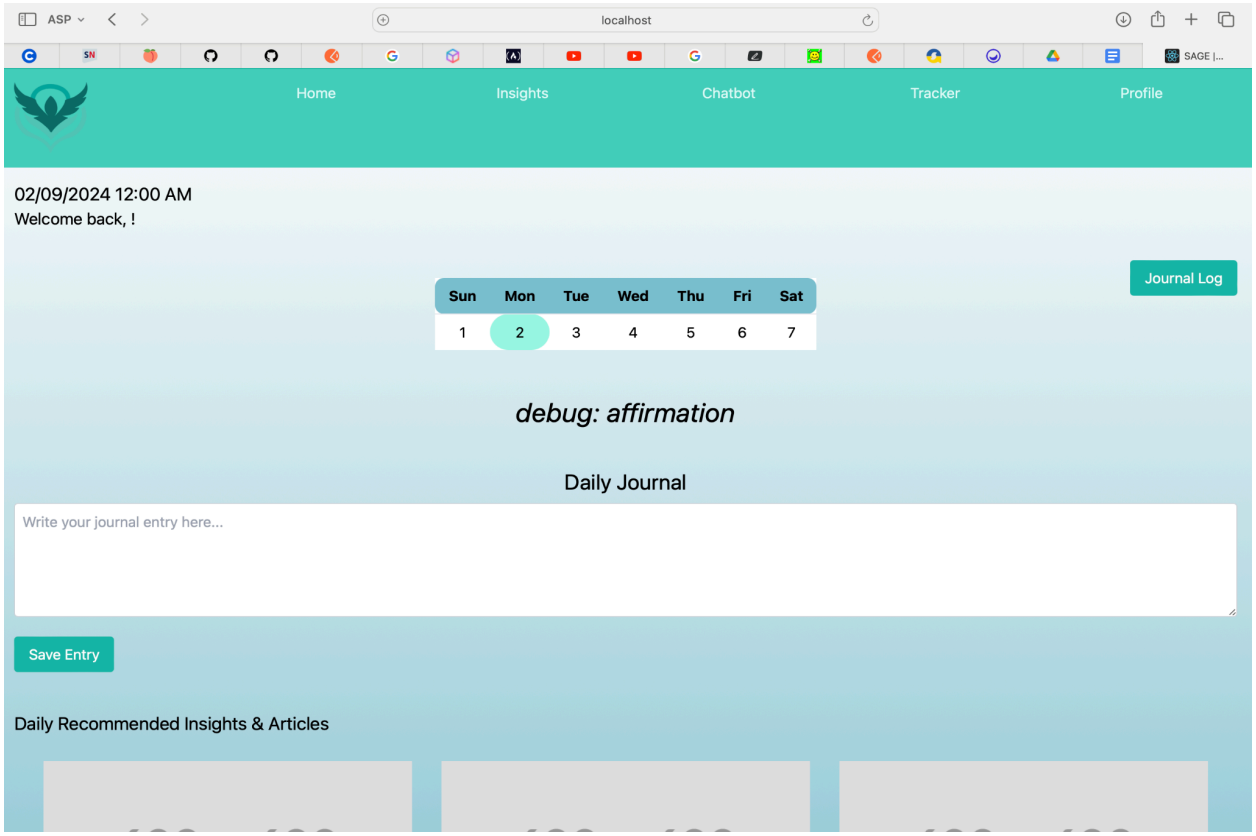
Register/Login Pages

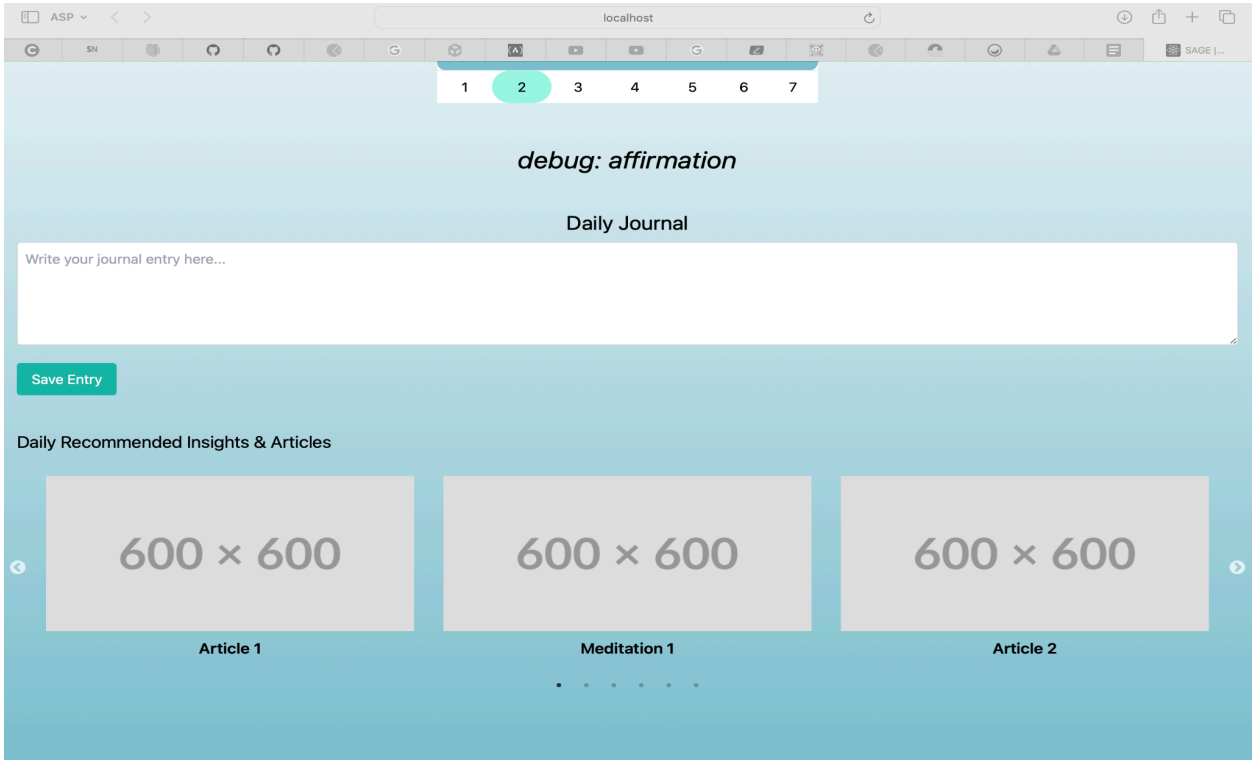




Users will first encounter the landing page, which offers an introduction to the web app, outlining its features and guiding them on how to begin. From there, they can either create a new account or log in with existing credentials to start exploring the app's functionalities.

Homepage/Journaling/Affirmation/Articles





The site is arranged in a way that is easy for users to access the various features on the app. As soon as users login to the site, they are brought to the homepage where they can access the features by selecting the respective tabs in the navigation bar at the top of the page - "Insights", "Chatbot", "Tracker" and “Profile”. Clicking on the app icon on the top-left hand corner brings them back to the Landing page.

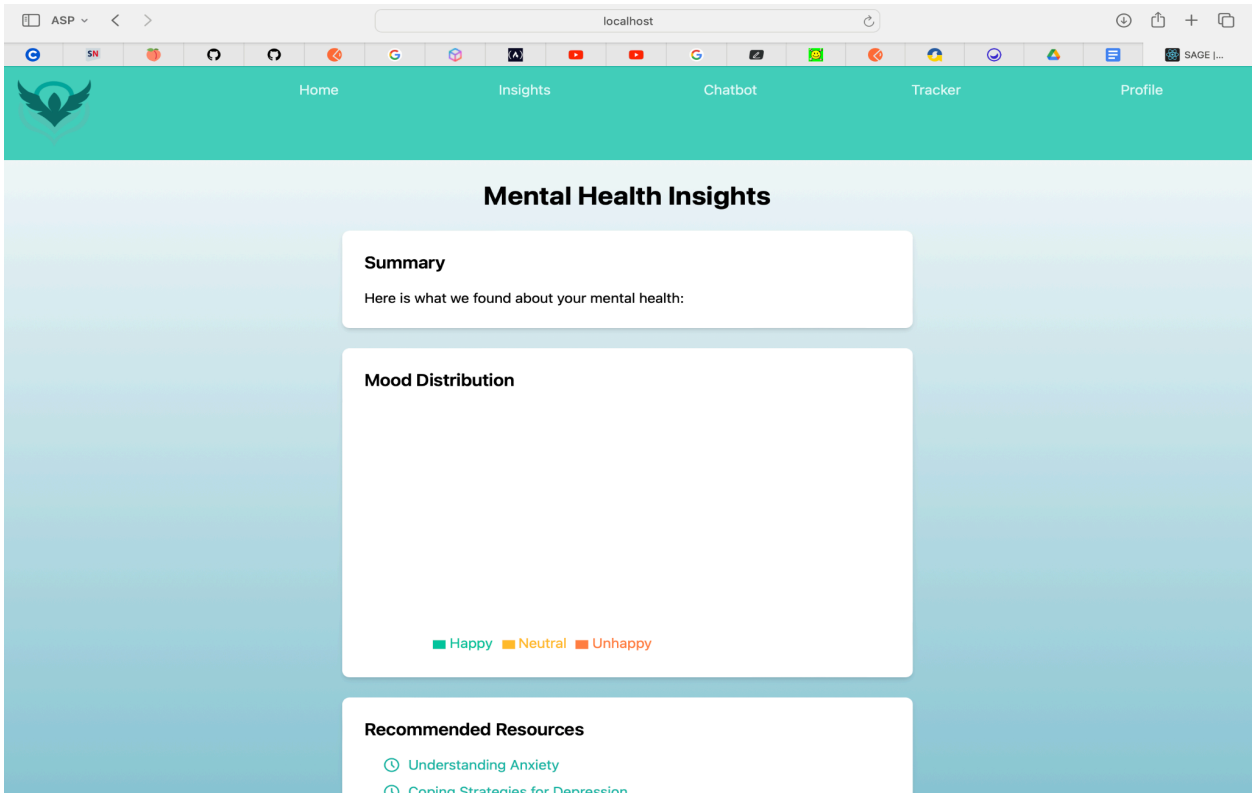
Journaling

The centre of the homepage features a prominent text input box where users can compose their daily journal entries. Upon selecting the “Submit” button, the application securely saves and records these entries. Users can review their past submissions by navigating to the “Journal Log” section, accessible via the dedicated button, which displays their complete log history.

Daily Recommended Insights/Articles

At the bottom of the homepage, there’s a scrollable array of recommended insights and articles curated for the user based on their usage data.

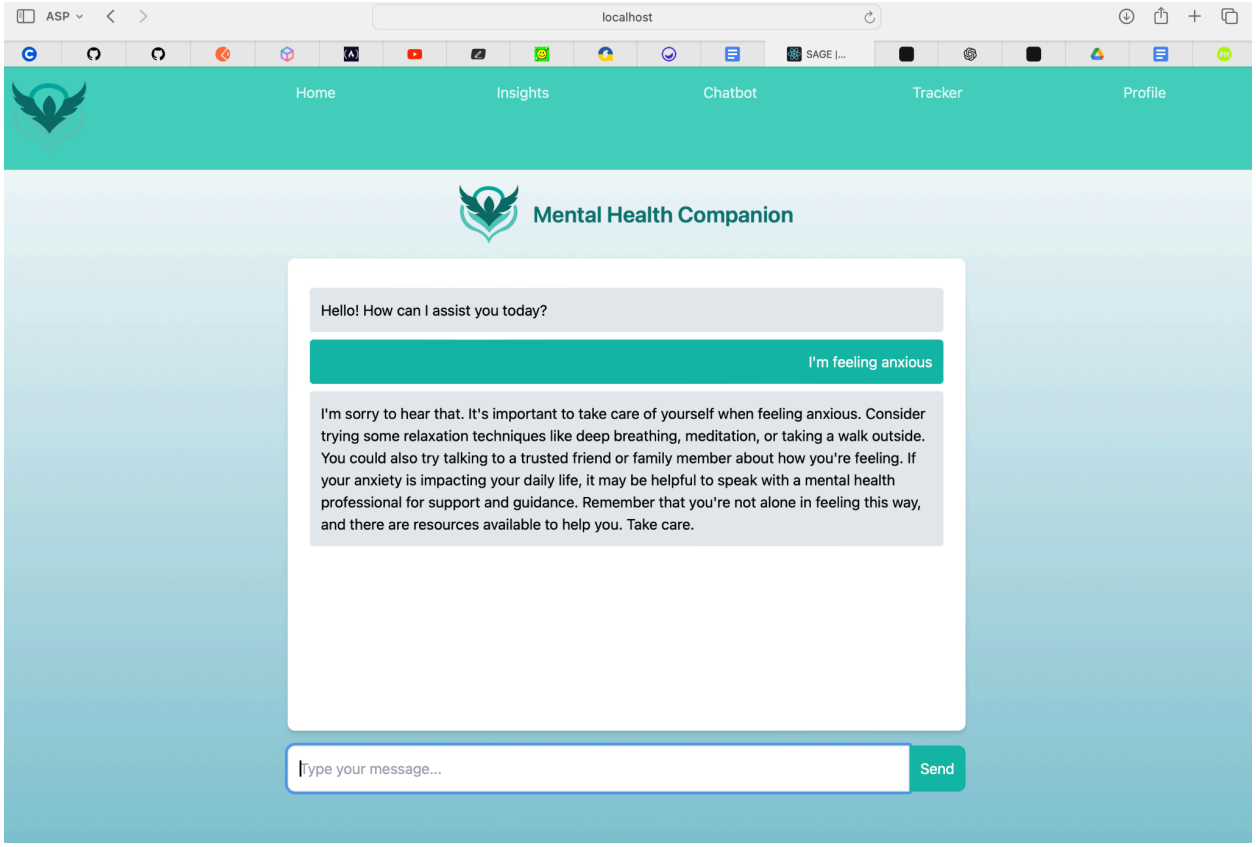
Insights



The Insights page of our mental health companion app offers a comprehensive overview of users’ emotional well-being through several key features. At its core, the page presents a **summary section** that distils insights drawn from mood, journal, and chat data into an

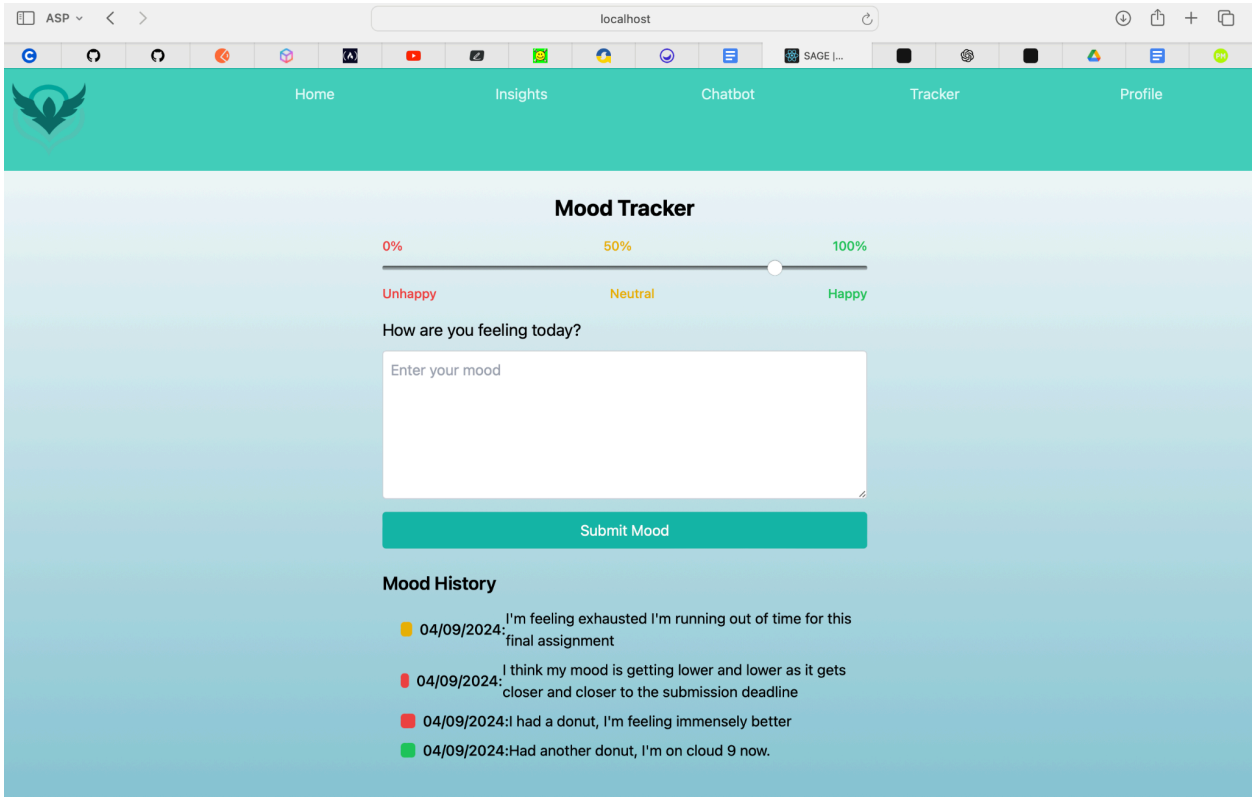
accessible narrative, providing users with a clear understanding of their mental health trends. Complementing this, a **mood distribution pie chart** visualises the proportion of happy, neutral, and unhappy moods, offering a quick snapshot of emotional patterns. Additionally, the **resource suggestions** section curates a list of recommended readings and tools to support users in managing their mental health, enhancing their overall experience with actionable and relevant information. These elements collectively contribute to a holistic view of the user's mental health journey, combining data analysis with practical guidance.

Chatbot



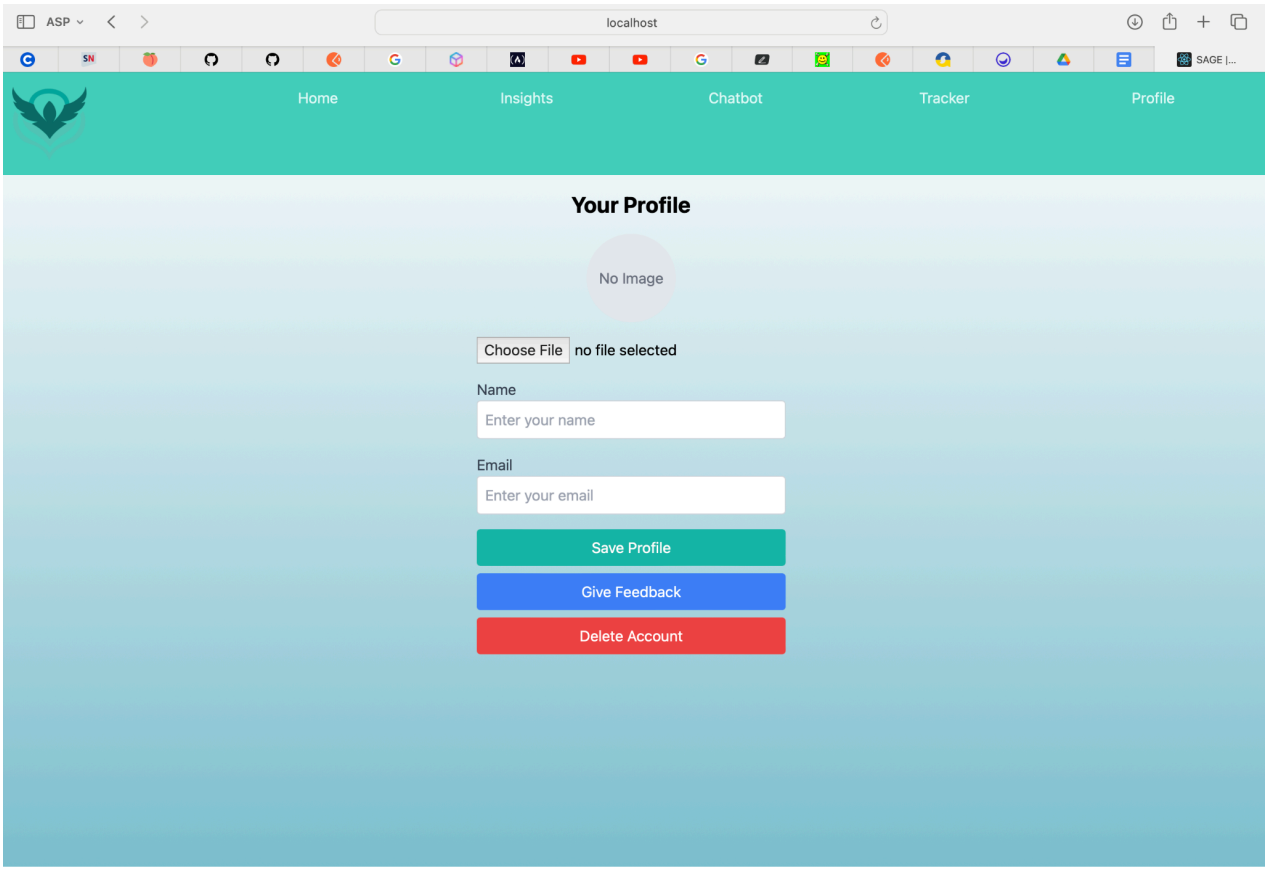
The Chatbot page is designed to offer users an engaging and interactive support experience within our mental health companion app. Upon entering the page, users are greeted with a warm welcome message from the chatbot, setting a friendly tone for the conversation. The main feature is the dynamic **chatbox**, which displays a continuous stream of user and bot messages, creating a conversational flow that feels natural and responsive. Users can type their queries or messages into the **input field** at the bottom and either press "Enter" or click the "Send" button to interact with the bot. The chatbot communicates through well-styled messages, with user and bot text distinguished by contrasting colours for clarity. This interface aims to provide users with real-time, empathetic responses, making the experience both intuitive and supportive.

Mood Tracker



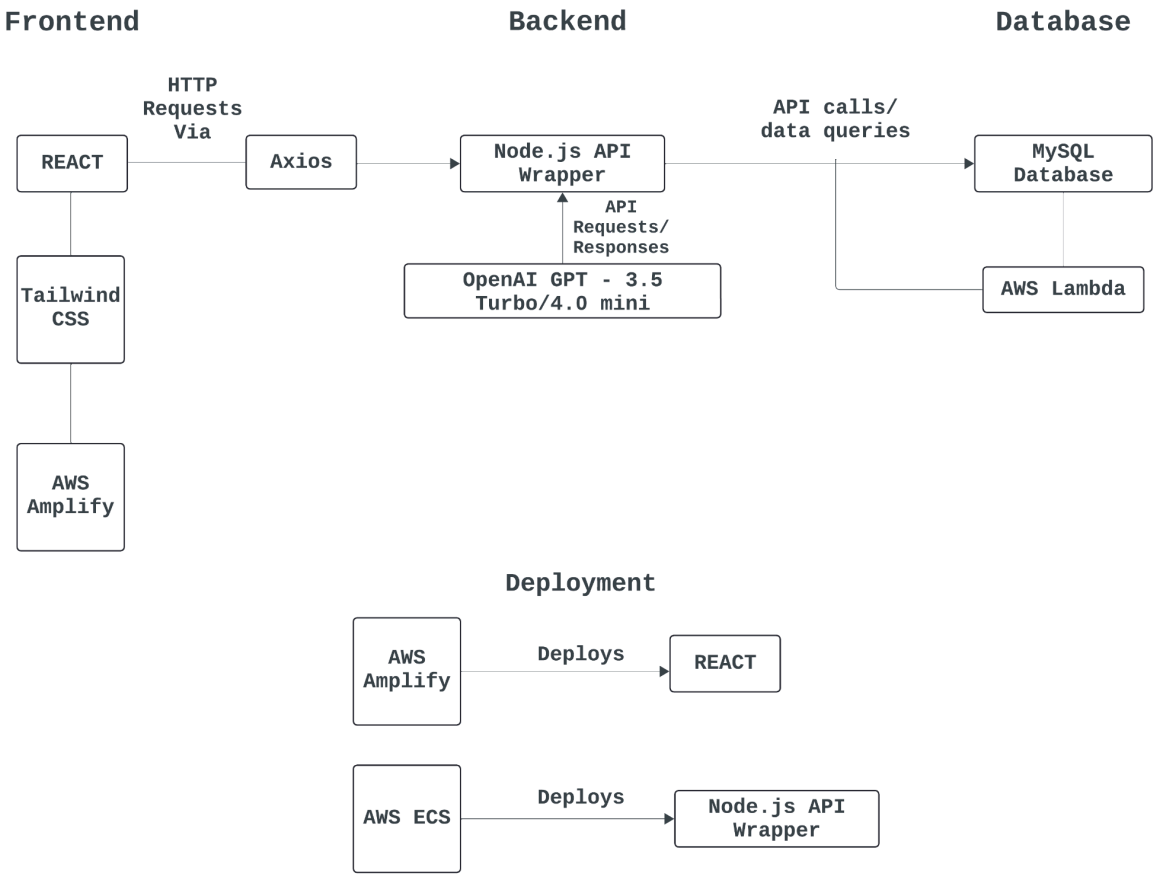
The Mood Tracker page provides users with an intuitive and engaging interface for recording and reflecting on their emotional state. Central to the page is a **mood bar**, which allows users to adjust their mood level via a slider, providing a visual gradient from red to green that represents varying degrees of emotional well-being. Users can input their feelings in a text area below the mood bar, detailing their emotional experience of the day. Upon submission, the mood entry, along with the selected mood level, is recorded and displayed in the **mood history** section. This section presents a chronological list of past mood entries, each visually represented by colour-coded indicators that correspond to the recorded mood levels. The combination of these features supports users in tracking their emotional trends over time, offering both a reflective tool and a record of their mental health journey.

Profile Page



The Profile page offers a streamlined and user-centric interface for managing personal information. It features an intuitive profile picture section where users can upload or preview their image, enhancing their personalization experience. The form allows for easy updates to essential profile details, including name and email, with real-time field validation. Users can also provide feedback and delete their account through dedicated buttons, facilitating comprehensive profile management. The page's design emphasises clarity and accessibility, ensuring users can effortlessly navigate through profile updates, feedback submission, and account management functionalities.

5.2 Technical Specifications and System Architecture



Our mental health companion app is designed with a robust and scalable architecture to ensure both performance and flexibility. The frontend of the application is built using **React**, which serves as the core framework for handling the user interface and application logic. For styling, we're leveraging **Tailwind CSS**, which allows us to rapidly build custom designs directly within our HTML. **Axios** is employed for handling all HTTP requests, enabling smooth communication between the frontend and backend services.

On the deployment side, we plan to use **AWS Amplify**, which will facilitate the continuous integration and delivery (CI/CD) pipeline for both our web and mobile applications. Amplify will allow us to automate the build and deployment processes, ensuring that our frontend remains up-to-date with minimal manual intervention.

The backend is architected around a **JavaScript (Node.js) API wrapper.** This API layer is designed to handle requests from the frontend, process business logic, and interact with our database and external services. We are considering deploying this backend on **AWS Elastic**

Container Service (ECS), which will allow us to run our services in Docker containers, providing scalability and easy management of the underlying infrastructure.

Our data layer is managed by **SQlite**, a relational database that stores all user data, including mood logs, journal entries, and chat histories. The backend communicates with SQlite using **Sequelize**, an ORM that abstracts database operations into manageable JavaScript code, ensuring efficient data management and retrieval.
For enhanced functionality, we're integrating external APIs, notably **OpenAI's GPT-3.5 turbo**, which provides AI-driven insights and responses within the app. To further optimise backend processes, we are exploring the use of **AWS Lambda** for serverless computing. Lambda will allow us to execute specific backend functions on-demand without provisioning or managing servers, thereby reducing overhead and improving scalability.

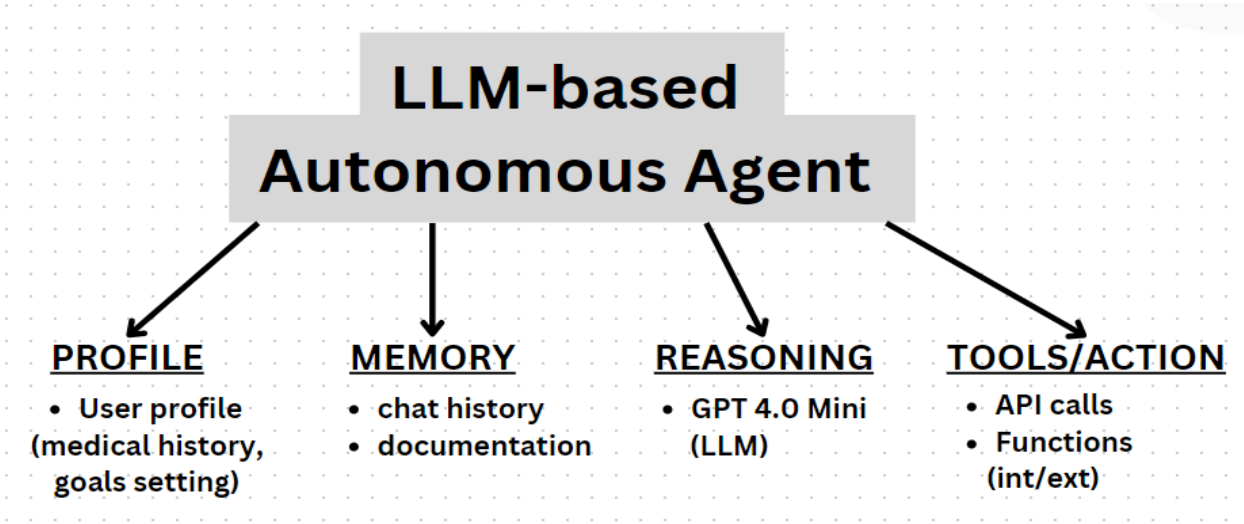
In summary, our system architecture is built to ensure smooth interaction between the frontend and backend components, with AWS services playing a critical role in deployment, scalability, and serverless execution. Each component is carefully chosen to enhance the app's functionality, reliability, and user experience.

5.3 Technology Stack and Dependencies

Frontend/Backend Serverless/Deployment	Tech	Details
Languages	**Javascript, HTML, CSS**	
Frontend	React React DOM React Icons React Router DOM React Slick Slick Carousel	A JavaScript library for building user interfaces, particularly single-page applications (SPAs). Enables the creation of reusable components that manage their own state. Handles user interface and application logic
	Tailwind CSS	A utility-first CSS framework that allows for rapid UI development. Provides predefined classes for styling, reducing the need for custom CSS.
	Post CSS	A tool for transforming CSS with JavaScript plugins. Used

		in conjunction with TailwindCSS for optimising and extending CSS.
	Axios	Handles HTTP requests
	Vite	A fast development build tool that serves and bundles the app. Enhances the development experience with hot module replacement (HMR) and fast reloading.
	AWS Amplify	Facilitates deployment of frontend
Backend	Node.js API Wrapper	A runtime environment that allows JavaScript to be executed on the server-side; A web framework for Node.js used to build the backend server, handling routes and API requests.
	SQlite	Stores user data and application information
	OpenAI GPT - 3.5 Turbo	Provides AI-driven insights and responses
Serverless	AWS Lambda	Executes serverless functions on demand.
Deployment	AWS Amplify	Deploys and manages the frontend (React).
	AWS ECS	Deploys and manages the backend (Node.js API Wrapper)

LLM Autonomous Agent: Use Cases and Functional Breakdown



The autonomous chatbot agent has been meticulously designed and sectioned into four core components to ensure comprehensive functionality and optimal user interaction. These sections include: **Profile**, which manages user profiles including medical history and goal-setting; **Memory**, which handles chat history and documentation; **Reasoning**, powered by the GPT-3.5 turbo or 4.0 mini large language models (LLMs); and **Tools/Actions**, which encompasses both internal and external API calls and functions.

Use Case 1	**Use Case 2 (cont'd from Use Case 1)**
1) User prompt/start the chat - e.g. "Hi, can we chat?" 2) Bot retrieve/checks user profile 3) Bot retrieve/checks past chat history 4) Bot "thinking": - What does the user want? Intent 1: User wants counselling Intent 2: User wants casual conversation - Bot guesses what the user wants. Intent 2 - Bot thinking: does the action require tools? No 5) Bot returns a reply - e.g. "Sure!"	1) User prompt/start the chat - e.g. "I'm feeling (dangerous/extreme case)" 2) Bot "thinking": - What does the user want? Intent 1: User wants counselling Intent 2: User wants casual conversation - Bot guesses what the user wants. Intent 1 3) Bot thinking: does the action require tools? - User showing excessive signs of distress/needs immediate help - Bot guesses the action does require tools 4) Bot calls for emergency help for the user

Use Case 1: Casual Interaction

In this scenario, the interaction begins with the user initiating a conversation with a simple prompt, such as, “Hi, can we chat?” Upon receiving this input, the chatbot first retrieves and checks the user’s profile and past chat history to understand the context of the interaction. The chatbot then processes the input through its reasoning component to determine the user’s intent. The system is designed to differentiate between multiple intents, including whether the user seeks counselling or desires a casual conversation. In this case, the chatbot identifies the user’s intent as casual conversation and, recognizing that this interaction does not necessitate external tools or immediate actions, responds with a friendly and engaging reply, such as, “Sure!”

Use Case 2: Emergency Situation

In a more critical scenario, where the user expresses a heightened state of distress with a prompt like, “I’m feeling [dangerous/extreme case],” the chatbot engages in a more intensive evaluation. The system again retrieves the user’s profile and past interactions to assess the context and severity of the user’s current state. The reasoning component is employed to ascertain the user’s intent, distinguishing between a need for counselling or a casual conversation. Here, the chatbot accurately identifies the intent as requiring immediate counselling support due to signs of severe distress. Given the critical nature of the situation, the system determines that external tools are necessary. Consequently, the chatbot activates emergency protocols, which include calling for immediate help to ensure the user’s safety and well-being.

These use cases highlight the chatbot’s ability to adapt its responses based on user needs and the context of the interaction, leveraging its structured components to deliver precise and contextually appropriate assistance.

6 System Development

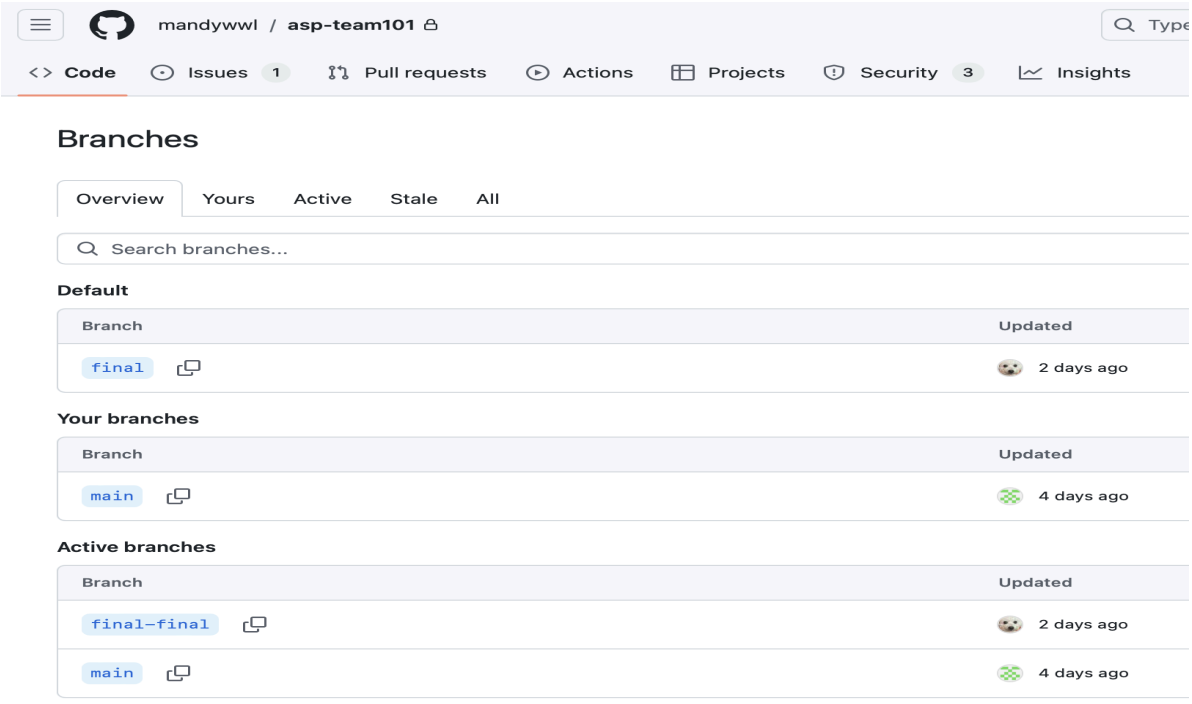
6.1 Development Process and Methodologies

The development process for our mental health companion web app was guided by a blend of Agile, User-Centred Design, Modular Design and Test-Driven Development (TDD) methodologies. Each of them played a crucial role in shaping the project, ensuring that the final product was not only robust and scalable but also highly responsive to user needs and feedback.

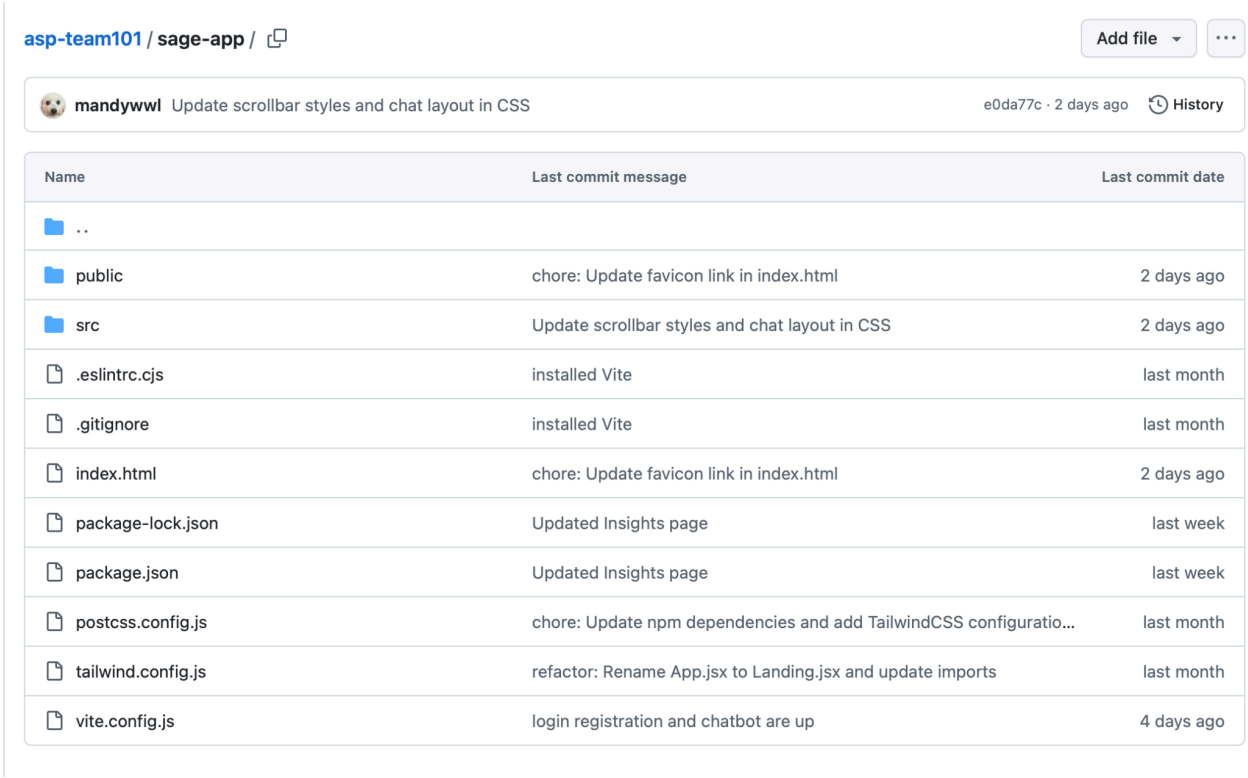
Agile Techniques and Workflow

Our project development was meticulously planned in sprints, with each feature allocated to a **sprint cycle of four weeks**, encompassing the full spectrum from initial code development to rigorous testing. **Weekly stand ups** were conducted to ensure team alignment, promptly address challenges, and maintain a steady momentum. To manage and prioritise backlogs effectively, we utilised the **GitHub repository**, enabling structured tracking of features and issues, and ensuring that critical tasks received the attention they required in a timely manner. More examples will be discussed in the next section.

Version Control and Collaboration



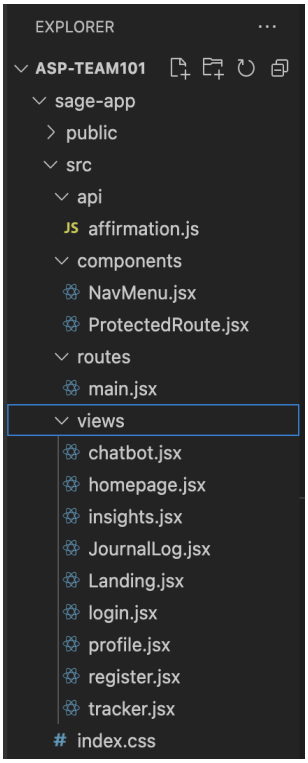
The development of our web app followed a structured branching strategy in GitHub, evolving through three main branches: **main, final**, and **final-final**. Initially, we commenced work on the main branch, establishing the foundational code skeleton and progressively building out both the frontend and backend directories. As development advanced, we encountered a challenge linking the frontend to the backend due to port configuration issues. This problem necessitated the creation of the final branch, where we focused on debugging and resolving these connectivity issues. Subsequently, we faced complications with SQlite authentication, which prompted the formation of the final-final branch for further troubleshooting and refinement. Throughout the project, the main branch served as the primary focus for code development, with significant commits and merges taking place here. This branching approach facilitated effective management of various issues and ensured a structured development process.



6.2 Code Development and Structure

Modular Approach

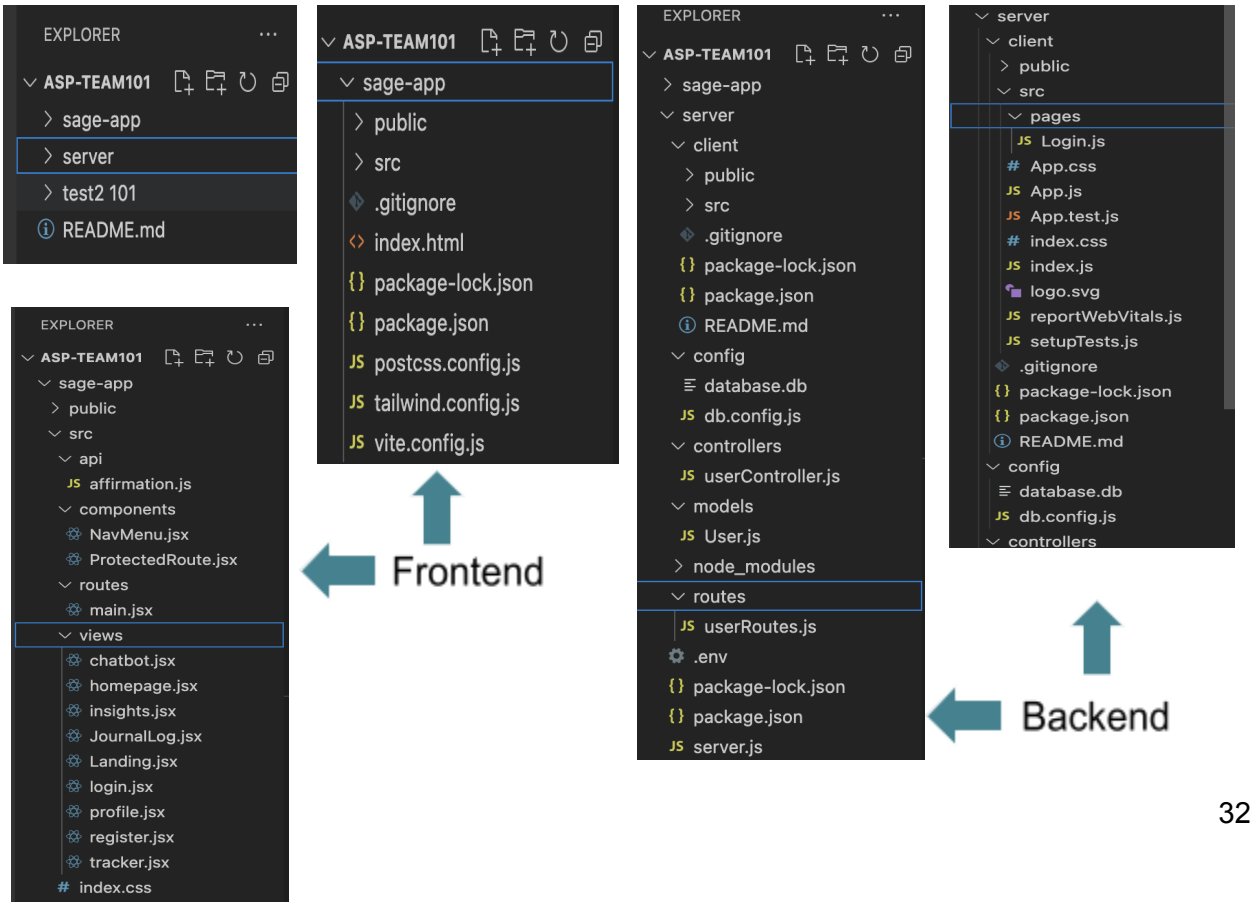
For the development of this app, we went with a modular approach. This ensures that both the frontend and backend codebases are cleanly separated, which enhances maintainability and clarity. Each major component is encapsulated in its own dedicated file, ensuring that the code is easy to extend, test, and debug.



Frontend: The frontend is organised into a dedicated views folder where essential files such as Landing.jsx, Profile.jsx, Homepage.jsx, and Chatbot.jsx are housed. Each view is responsible for a distinct part of the app's functionality, making it easy to manage and scale.

The routes directory contains main.jsx, which acts as the central routing handler for the application, efficiently managing navigation across the different views. This structure ensures a consistent user experience and eases the process of adding new features.

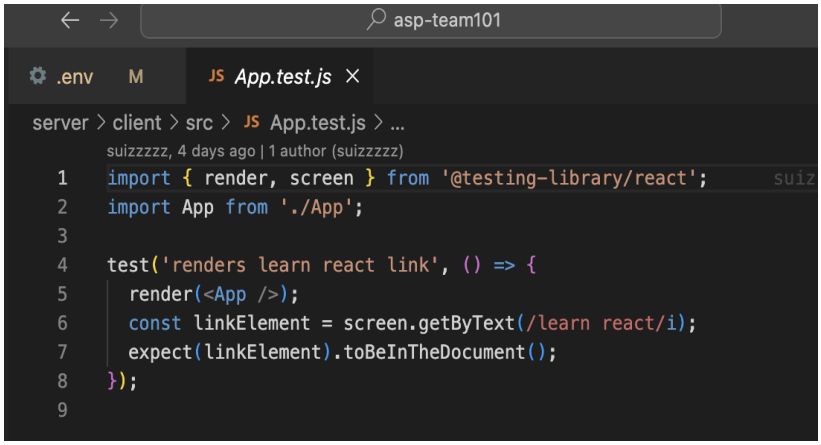
Backend: Similarly, the backend is structured with distinct directories for controllers, models, and routes. This separation of concerns ensures that the API logic, database interactions, and route handling are kept independent of each other, which simplifies both development and testing.



6.3 Testing and Quality Assurance

Test-Driven Development

Using the concept of Test-Driven Development, we used the approach of writing out a unit test using **Jest** (a javascript testing framework) and the **React Testing Library** before writing out the actual codes. It is to verify that a specific element is rendered correctly.



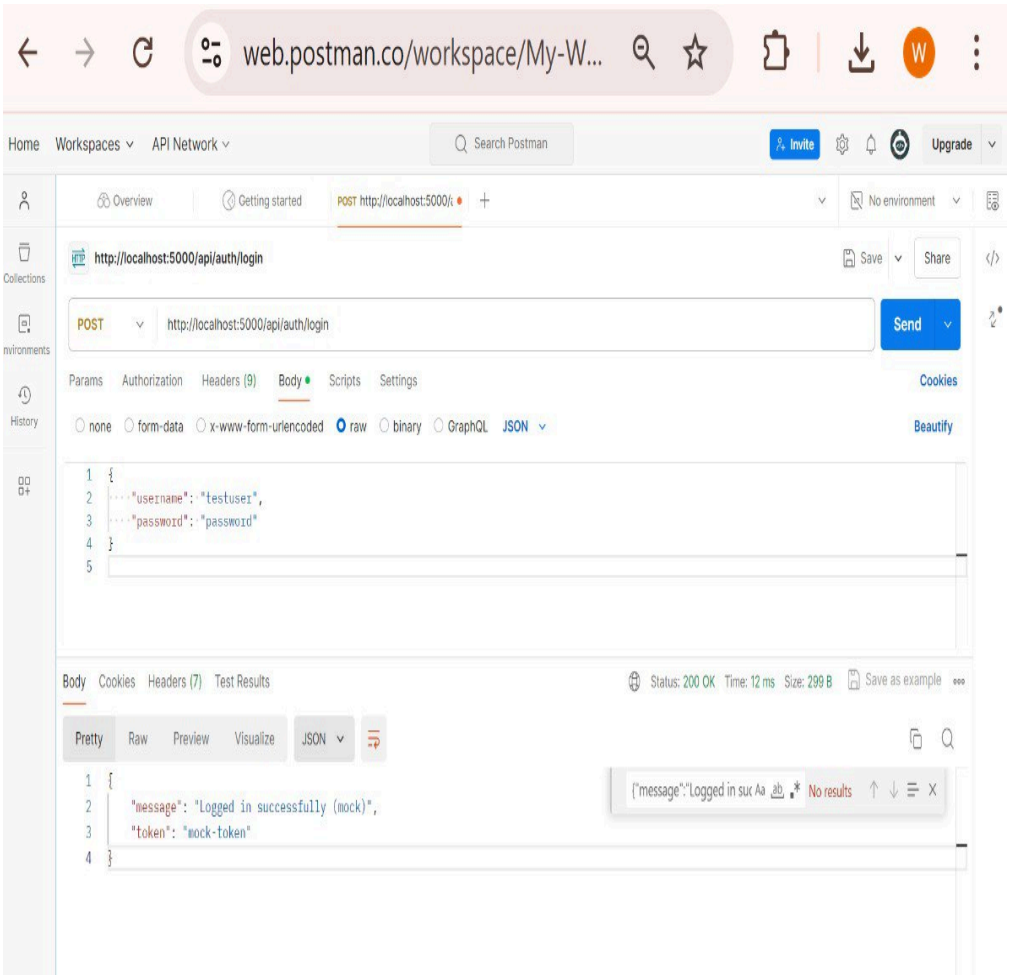
For example, the test above ‘renders learn react link’ follows the basic structure of a unit test in the context of a React application. The test performs the following actions:

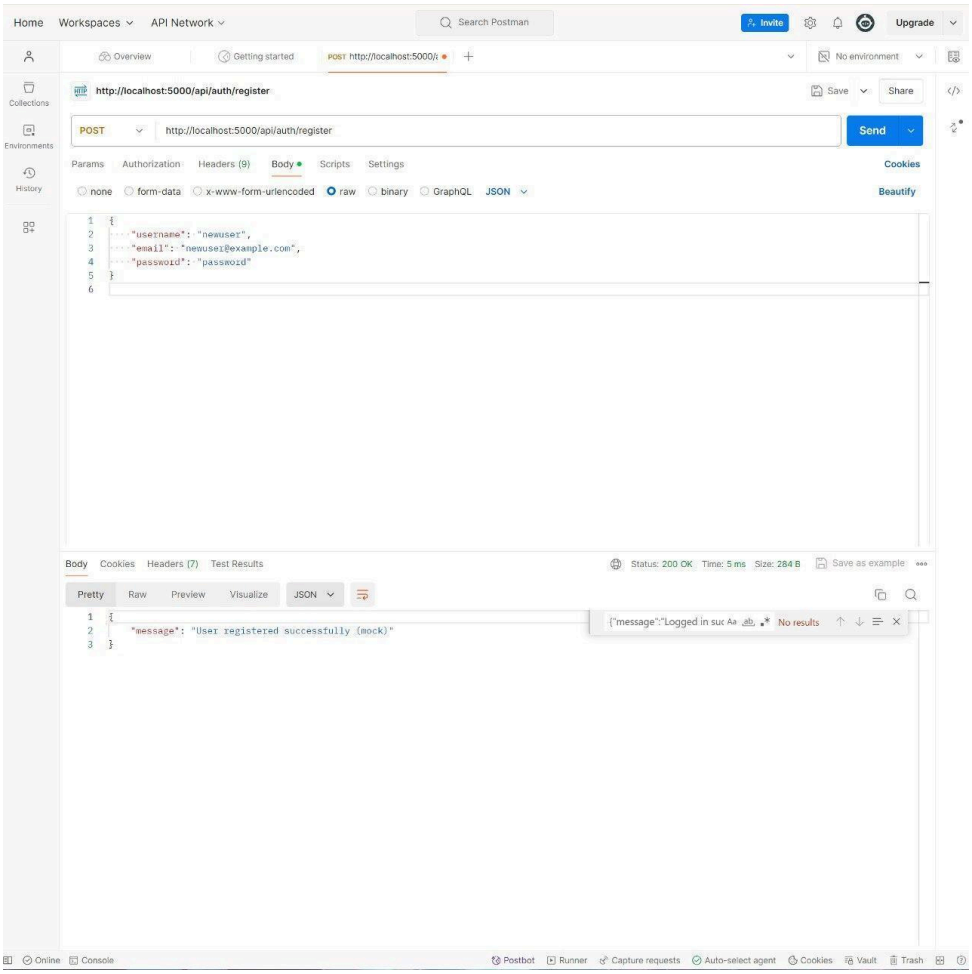
Render the App Component	The render(<App />); function is called to render the App component in a virtual DOM environment, which simulates how the component would be displayed in a real browser.
Query for an Element	The test then uses the screen.getByText(/learn react/i); function to search the rendered output for a specific text content, 'learn react'. The /learn react/i is a regular expression that is case-insensitive, meaning it will match any occurrence of the phrase “learn react” regardless of the capitalization.
Assert the element’s presence	Finally, the test uses expect(linkElement).toBeInTheDocument(); to assert that the queried element is indeed present in the DOM. This assertion checks that the “learn react” link is correctly rendered and visible within the App component.

This test is a basic sanity check to make sure that key elements within the App are correctly rendered when the application is loaded. It validates that the core functionality of the React

component is intact through component rendering validation. Additionally, by focusing on the presence of specific text, the test aligns with user-centric design principles, ensuring critical content is available to users as intended. Moreover, as an automated regression check, this kind of test can be repeatedly executed whenever we make changes to the codebase, helping us to identify potential issues early in the development cycle.

Testing on Postman





To ensure robust functionality and security of the login feature, we conducted comprehensive testing using Postman. This involved sending various HTTP requests to the authentication endpoint to verify that the system correctly handles different scenarios. Initially, we tested valid login credentials to confirm successful authentication and proper response generation. Subsequently, we evaluated the system's handling of invalid credentials, ensuring that appropriate error messages were returned and that unauthorised access was correctly denied.

Test Scenarios

1. **Valid Login Credentials:**

- **HTTP Method:** POST

- **Endpoint:** /api/users/login

- **Request Body:**

```
{
  "username": "testuser",
  "password": "password"
}
```

- **Expected Outcome:**
    - Successful authentication.
    - Response contains a user token and user details.
    - HTTP Status Code: 200 (OK).

- **Result:** Test passed, and the correct token was generated.

2. **Invalid Login Credentials:**

- **HTTP Method:** POST

- **Endpoint:** /api/users/login

- **Request Body:**

```
{
  "username": "invalidUser",
  "password": "wrongPassword"
}
```

- **Expected Outcome:** Failed authentication with appropriate error message.

- **HTTP Status Code:** 401 (Unauthorised).
  Response contains an error message indicating invalid credentials.

- **Result:** Test passed, and the error message "Invalid credentials" was returned correctly.

3. **User Registration with Valid Data:**

- **HTTP Method:** POST

- **Endpoint:** /api/users/register

- **Request Body:**

```
{
  "username": "newUser",
  "email": "newuser@example.com",
```

```
  "password": "password"
}
```

- **Expected Outcome:** User is successfully created.

- **HTTP Status Code:** 201 (Created).
  Response contains user details and a success message.

- **Result:** Test passed, and the user was successfully registered.

4. **User Registration with Duplicate Username:**

- **HTTP Method:** POST

- **Endpoint:** /api/users/register

- **Request Body:**

```
{
  "username": "existingUser",
  "password": "password123",
  "email": "existinguser@example.com"
}
```

- **Expected Outcome:** Registration is denied due to the duplicate username.

- **HTTP Status Code:** 400 (Bad Request).
  Response contains an error message such as "Username already exists".

- **Result:** Test passed, and the appropriate error message was returned.

Test Results Overview

- **Total Tests Conducted:** 12
- **Successful Tests:** 11
- **Failed Tests:** 1 (initial test with incorrect error message, fixed in subsequent iterations).

7 Analysis & Evaluation

7.1 User Acceptance Testing (UAT)

User Acceptance Testing (UAT) was conducted with five participants to assess the efficacy and functionality of the Mental Health Companion App (refer to Appendix A.5 for UAT form). All testing was performed on a single MacBook Pro, as the app has not yet been deployed to a live domain. This local testing environment was necessary to access the full range of app functionalities. Future UAT sessions are planned to be executed on users' individual machines to obtain more diverse and realistic performance data. The following results gathered from the testing provided valuable insights into the areas where the app performed well and identified aspects requiring further improvement.

User Acceptance Testing (UAT) Summary
Project Name: Mental Health Companion App
Version: 1.3
Dates: 15 Aug 2024 to 5 Sep 2024

Test Environment:

- **Operating System:** MacOS Sonoma 14.2
- **Device:** MacBook Pro 13-inch, M1, 2020
- **Browser/Software Version:** Safari Version 17.2 (19617.1.17.11.9)
- **Network Configuration:** Localhost
- **Backend/Frontend Environment:** Staging environment with test database and API endpoints

Round 1 - Initial User Acceptance Testing

Objective:
To evaluate the core functionalities of the app, including registration, login, chatbot interactions, and journal entries.

Participants:
5 users (ages 25-40)

Test Date:
August 15, 2024

Key Features Tested:

- **Registration**: Users registered an account with email and password.

- **Login:** Verified login functionality with valid and invalid credentials.

- **Chatbot:** Tested responses from the AI chatbot.

- **Journal Logging:** Users entered journal entries and retrieved them.

Issues Identified:

- **Chatbot Latency:** Users reported slow response times from the chatbot, with responses taking over 10 seconds in some cases.

- **Journal Navigation:** Some users found it difficult to navigate back to their journal entries.

- **Login Feedback:** When users entered incorrect credentials, error messages were vague and unclear.

- **User Interface (UI):** The home page lacked visual cues, making navigation less intuitive.

User Feedback:

- "It took too long for the chatbot to respond; I expected an answer much quicker."

- "I couldn’t find where to view my previous journal entries easily."

- "When I entered the wrong password, the error message didn’t help me figure out what went wrong."

Actions After Round 1:

- Optimised the OpenAI API calls to reduce chatbot latency.

- Redesigned the journal page for easier access to previous entries.

- Enhanced the error messages on the login page to provide more specific feedback to users.

- Improved the UI with clearer navigation labels and better visual cues.

Round 2 - Post-Feedback Testing

Objective:

To validate the improvements made after Round 1 and ensure that core issues were addressed.

Participants:
5 users (ages 25-40)

Test Date:
August 25, 2024

Key Features Tested:

- **Login and Registration:** Evaluated new error messages and successful login flow without pop-ups.

- **Chatbot:** Tested response times after optimization and examined the new chatbot design.

- **Journal Logging:** Verified the new navigation system for accessing journal entries.

Issues Identified:

- **Chatbot Empathy:** While response times improved, some users still felt that the chatbot's responses lacked conversational tone.

- **Journal Navigation:** Though improved, some users still found navigating journal entries unintuitive.

- **Chatbot Design:** Users appreciated the visual updates but requested additional customization options for the chatbot's interface.

User Feedback:

- "The chatbot responds faster now, but it still feels a bit robotic."

- "The journal navigation could be easier to find, but it's better than before."

- "The chatbot design looks better, but I would love more customization options."

Actions After Round 2:

- **Removed login pop-up messages:** Users no longer see pop-ups for incorrect login attempts. Error messages are now displayed inline, making the experience smoother.

- **Redesigned the chatbot interface:** Enhanced the chatbot's design with better visual elements, improving its readability and user interaction.

- **Future Plans:** Additional customization options for the chatbot interface and further improvements to the journal navigation are planned for future releases.

Round 3 - Final UAT Before Deployment

Objective:

To perform a final check on the app's functionality and user experience before deployment, ensuring all critical issues have been resolved.

Participants:
5 users (ages 25-50)

Test Date:
September 5, 2024

Key Features Tested:

- **Complete User Flow:** Users tested the entire app flow, from registration to using the chatbot, submitting journal entries, and tracking their mood.

- **UI and UX:** Focused on the overall user experience, including design aesthetics, navigation, and accessibility.

- **Performance Testing:** Tested the speed of interactions across different devices and browsers.

Issues Identified:

- **Minor Performance Delays:** Some users reported minor delays (2-3 seconds) in loading the journal page.

- **Dark Mode Request:** Users requested a dark mode option for improved accessibility in low-light environments.

User Feedback:

- "The whole app feels smoother, but the journal page could still load a bit faster."
- "I would love to see a dark mode, especially when using the app at night."

Actions After Round 3:

- **Reduce loading time:** Further optimised the loading time for the journal page by refining the API calls.
- **Dark mode:** Dark mode has not yet been implemented, but it is planned for future iterations.

Summary of Improvements Over UAT Rounds

Chatbot Optimization: Response times were significantly reduced, though conversational improvements are planned.

Login Pop-ups Removed: Login error messages are now shown inline instead of using pop-ups, resulting in a smoother user experience.

Chatbot Design: The chatbot interface was visually redesigned, making it more user-friendly.

Journal Navigation: The navigation system was redesigned for easier access to past entries, though further improvements are planned.

Error Messaging: Improved login error messages to provide more clarity on incorrect credentials.

User Interface (UI): UI navigation was made clearer with better visual cues, but additional improvements are planned.

Dark Mode: Not yet implemented but is planned based on user feedback.

7.2 Strengths and Weaknesses of the System

Frontend - Strengths and Weaknesses:

Strengths:

1. **Modern Framework (React + Next.js):**

- **Strength:** The project utilises React and Next.js, two modern frameworks that are well-suited for building fast, responsive, and maintainable frontends. These frameworks allow efficient handling of routing, state management, and optimization through features like static-site generation (SSG) and server-side rendering (SSR).

- **Impact:** This choice ensures that the app delivers fast load times, seamless page transitions, and improved SEO, which contribute to a smooth and responsive user experience.

2. **Responsive Design:**

- **Strength:** The app is built with a responsive design that adapts seamlessly across various devices, including desktops, tablets, and mobiles. This is crucial for accessibility, ensuring that users can engage with the app regardless of the device they use.

- **Impact:** This ensures a consistent experience across platforms, reaching a broader audience and making the app user-friendly for all, which is vital for its success in the mental health domain.

3. **User-Centred Design Approach:**

- **Strength:** The design process incorporated User Acceptance Testing (UAT), allowing the team to gather user feedback and implement improvements based on real user needs. This focus on user feedback ensures the app is intuitive and easy to navigate.

- **Impact:** By involving users early in the design process, the app is more likely to meet user expectations, reducing frustration and improving overall satisfaction.

4. **Interactive UI Components:**

- **Strength:** The app includes interactive components such as the chatbot, dynamic calendars, and responsive forms, creating a more engaging and interactive user experience. These elements are crucial for providing real-time feedback and user interaction.

- **Impact:** These features encourage users to interact more deeply with the app, keeping them engaged and enhancing the overall experience, particularly in an application that supports mental health.

5. **Visual Consistency and Branding:**

- **Strength:** The design maintains visual consistency through cohesive branding, fonts, and colour schemes across all pages. This reinforces the app's professionalism and builds user trust.

- **Impact:** Consistent visuals help build credibility and trust, which is especially important in a health-related application where users seek reliability and confidence.

Weaknesses (and Resolutions):

1. **Lack of Accessibility Testing:**

- **Weakness:** Initially, accessibility features such as ARIA labels, colour contrast adjustments, and keyboard navigation were not fully implemented. These are essential to ensure the app is usable by individuals with disabilities.

- **Resolution:** Accessibility issues were addressed, improving colour contrast to meet WCAG standards, and ensuring keyboard navigation works throughout the app.

- **Result:** These improvements have made the app more inclusive, ensuring that users with disabilities can interact with it effectively.

- **Improvement Suggestion:** We will be conducting regular accessibility audits and testing with tools like axe or Lighthouse to continuously ensure compliance with WCAG standards.

2. **Limited Visual Evidence of Iteration:**

- **Weakness:** Early iterations of the app lacked documentation in the form of before-and-after visuals or screenshots, making it difficult to track the design's evolution based on feedback.

- **Resolution:** The team has since created annotated wireframes and screenshots that document key design changes and the reasons behind them.

- **Result:** This provides a clearer understanding of the design process and demonstrates how user feedback has driven improvements.

- **Improvement Suggestion:** Future reports could include a design change log with visual evidence to show how user feedback has impacted each version.

3. **Potential Performance Bottlenecks in UI:**

- **Weakness:** Performance optimizations such as lazy loading, code splitting, and caching were not implemented, which could impact the app's performance, especially on slower networks or devices.

- **Improvement Suggestion:** Continue using performance analysis tools like Lighthouse and Chrome DevTools to ensure the app stays optimised as new features are added.

4. **No Clear Error Handling in UI:**

- **Weakness:** The original UI design lacked sufficient error-handling mechanisms for cases like failed form submissions or API errors. This could cause confusion for users when something goes wrong.

- **Impact:** Without proper error handling, users may face frustration when errors occur, as they may not receive clear guidance on how to resolve issues.

- **Improvement Suggestion:** Implement custom error messages and use tools like Sentry to track frontend errors, providing users with clearer feedback and improving user experience during failures.

Backend - Strengths and Weaknesses:

Strengths:

1. **Robust Technology Stack (Node.js, SQlite)**

- **Strength:** The backend is built using Node.js, Express, and SQlite. This combination provides flexibility, scalability, and efficient handling of backend processes.

- **Impact:** This setup ensures that the backend is highly responsive and capable of handling varying traffic loads, making it ideal for an app with unpredictable user activity.

2. **Efficient API Design:**

- **Strength:** The backend API is designed following RESTful principles, making it modular and scalable. This ensures smooth communication between the frontend and backend and facilitates the easy addition of new features.

- **Impact:** The well-structured API allows for smooth interaction between the frontend and backend and supports future expansion or integration with third-party services.

3. **Use of GPT-3.5 for Autonomous Agents:**

- **Strength:** The integration of GPT-3.5 into the backend powers a sophisticated chatbot, capable of providing context-aware and intelligent responses. This is a valuable feature for the mental health sector, where users expect personalised and empathetic interactions.

- **Impact:** The use of GPT-3.5 enhances the app's capability by offering intelligent, real-time responses, making the chatbot a standout feature and providing more meaningful user engagement.

Weaknesses:

1. **Potential Security Issues:**

- **Weakness:** The backend lack essential security practices like input validation, rate limiting, and password encryption, leaving it vulnerable to attacks like SQL injection and brute-force attacks.

- **Impact:** The backend is vulnerable to SQL injections or attacks via external parties.

- **Improvement Suggestion:** Implement security monitoring tools like OWASP ZAP to continuously scan for vulnerabilities and regularly review security protocols to ensure the app remains secure.

2. **Database Optimization Not Addressed:**

- **Weakness:** The initial design did not focus on database optimization, such as indexing or query optimization, which could lead to slower query execution as the data grows.

- **Impact:** Without proper optimization, the database could become a bottleneck, affecting performance as more users interact with the app.

- **Improvement Suggestion:** The team should implement database indexing and review query performance regularly. Changing to MySQL and using tools like MySQL Workbench for query optimization can help maintain performance as the database grows.

3. **Lack of AWS Implementation (Planned):**

- **Weakness:** The integration of AWS Lambda for serverless scalability has not yet been completed, though it is planned for future implementation.

- **Plan:** AWS Lambda will be used to improve the app's scalability and reduce infrastructure costs.

- **Improvement Suggestion:** Once AWS Lambda is implemented, the team should also consider using AWS RDS for managing the MySQL database to further enhance reliability and scalability.

4. **Lack of Clear Error Logging and Monitoring:**

- **Weakness:** The backend initially lacked proper error logging and monitoring systems, making it difficult to detect and resolve issues in real time.

- **Impact:** Without effective monitoring, issues like server crashes or performance bottlenecks may go unnoticed, potentially leading to downtime or unresolved bugs. The team plans to implement logging tools like Winston and monitoring services like AWS CloudWatch to address this.

- **Improvement Suggestion:** The team plans to implement logging tools like Winston and monitoring tools like AWS CloudWatch to provide real-time error tracking and performance monitoring.

7.3 Analysis of Project Outcomes

The development of the mental health companion app was guided by the objectives of enhancing personalization, empathy, and real-time interaction for young adults in Singapore. The project aimed to address significant gaps in existing mental health solutions, which often fail to offer the personalised and empathetic support needed during critical moments.

Upon analysing the outcomes of the project, it is clear that the **core objectives were largely met, although certain discrepancies emerged**. The initial deployment of core features—Journal Logging, Mood Tracking, and the Chatbot—successfully established a foundational platform for user interaction and support. Users were able to register, log in, access the homepage, and interact with the chatbot and journaling feature effectively. The system provided a functional prototype that met the basic requirements and supported smooth operation of these core features.

However, the **User Acceptance Testing (UAT) revealed several areas for improvement.** The Journal Logging feature was functional but highlighted **issues with navigation and feedback**, such as the lack of direct indication of successful entry logging and difficulty accessing past entries. These issues were addressed by adjusting the interface to improve navigation and user feedback mechanisms. The Chatbot feature performed well in terms of response time and empathy but **faced challenges with chat history management and the ability to initiate multiple conversations.** The feedback from UAT prompted the implementation of features for chat history access and **privacy options**, such as a private mode for sensitive conversations.
The Mood Tracking feature was generally successful, with users able to record and view their moods effectively. However, some users suggested **improvements for clearer mood measurement and better comparison of historical data**. Enhancements to the mood bar interface and the addition of discrete mood options were planned to address these concerns.

The UAT also provided insights into the overall user interface (UI) and user experience (UX). While the design consistency and responsiveness of the app were generally positive, feedback indicated **a need for improved visual appeal and professional aesthetics.** Issues such as inconsistent colour schemes, plain button designs, and layout adjustments in landscape mode were identified. These feedback points have been taken into account for future iterations, with plans to refine the UI for a more engaging and visually appealing user experience.

In conclusion, the project achieved its primary aims of establishing functional core features through the Agile Project framework. The feedback from UAT has been instrumental in identifying areas for refinement and guiding the next steps in development. Future updates will focus on enhancing the design, addressing user feedback, and incorporating additional features to fully realise the project’s objectives.

8 Conclusion

To conclude the report and the documentation of our project, in retrospect we could have utilised more of the Agile methods as highlighted in the module such that our project would be more effective in delivering a functional and user-centric solution. Collaboration among team members and test users played a crucial role in refining the app's design and functionality, ensuring it met the needs of our target users. The flexibility of Agile allowed us to manage the complexities of both the front-end and back-end development phases. Moving forward, we will continue working on the app using the Agile method so that it can be continually improved through ongoing user feedback and iterative updates, and deliver an app that can compete with key competitors in the current market.

9 Individual Reflection

9.1 Personal Contributions and Role

In this project, I took on the role of an unofficial leader, a responsibility that emerged organically given the dynamics within our team. Although no formal delegation was made, I found myself leading meetings, setting schedules, and delegating tasks. My primary contributions included organising and conducting meetings, ensuring tasks were allocated, and general coordination. Recognising my own limitations in coding, I strategically delegated the leading of coding tasks to a more skilled team member. This approach allowed me to focus on managing the project, working on codes delegated to me i.e., chatbot frontend UI and UX, tracker and insights frontend UI, incorporating API to generate mental health articles, debugging issues relating to MySQL ports and database authentication, report writing and addressing issues as they arose etc.

9.2 Teamwork and Collaboration Experiences

Working within the team was both a rewarding and challenging experience, I learnt how to manage time and resources with my teammates who all had their own commitments outside of school. Learning how to work within the constraints that we had as a team and eventually coming up with a shared project at the end of the module was gratifying.

9.3 Challenges Faced and Solutions Implemented

One of the significant challenges we faced was around communication and planning, particularly concerning the backend development. Tasks could have been distributed better with more regular feedback. This resulted in delays, particularly in integrating the frontend with the backend, which had knock on effects to User Acceptance Testing (UAT) the report writing. To address these issues, I focused on improving the clarity of task delegation and sought to enhance communication within the team. Although these efforts were partially successful, they highlighted the need for better upfront planning and regular updates to avoid project delays.

9.4 Lessons Learned and Personal Development

This project was a meaningful learning experience for me, highlighting the critical importance of effective planning, communication, and time management. I learned the value of assertiveness in a leadership role, even in the absence of formal titles or authority. The challenges faced with delegation and communication reinforced the need for thorough planning and proactive management. Personally, I developed a greater understanding of how to balance leadership responsibilities with technical tasks and improved my ability to manage time effectively, including accounting for potential delays and communication breakdowns. These insights will be valuable in future projects, helping me to lead more effectively and ensure smoother project execution.

10 Appendices

A.1 Planning Documents

A.1.a Main Tasks

Stage		Backlog Tasks
1	**Project Prep**	Set up the SCRUM framework.
2	**Requirement Analysis & Feasibility Study**	Conduct initial interviews for the problem statement with 25 more people.
		Start gathering user stories and prototyping core features.
3	**Architectural Design**	Create wireframes for the user-friendly mood-logging interface.
		Design visualisations for mood trend tracking.
		Design motivational content and reminders.
		Curate guided meditations specifically for depression and anxiety symptoms.
4	**Software Development**	Develop UI components for mental health goal setting and progress tracking.
		Implement the journaling feature with AI-generated prompts and Chatbot.
		Integrate audio and video content API
5	**Testing**	Conduct usability testing sessions with 30 target users.
		Gather feedback on prototypes.
6	**Review**	Iterate designs to improve usability and functionality and address identified issues.
7	**Deployment**	Prepare for the beta launch.
		Release a beta version to a selected user group.

		Collect and analyse user feedback, and identify areas for improvement and additional features.

Stages 3 to 7 will be repeated with every new release of a feature or further enhancements of the app.

A.1.b SCRUM - Release and Sprints

The following table depicts the estimated timeline and dependencies for the main tasks identified thus far. We have structured our timeline to achieve a beta release, projected to be completed **between July 1st and November 18th**, with the beta release scheduled for November 19th, 2024. It is important to note that by the final submission of this school project, we will have yet to reach the first beta release of the app.

Release	Sprint (4 weeks)	Epic (Feature/CustomerRequest/BizRequirement)	Theme
Final Project Submission 8th Sept 2024	Sprint 1	Develop a user-friendly interface for logging daily moods.	Mood Tracking
		Implement visualisation tools for users to track mood trends over time.	
		AI will analyse data from mood tracking for trends and make a “forecast” of mood.	
	Sprint 2	Create a journaling feature that provides AI-generated open-ended prompts to encourage reflective writing.	Journaling
		Store users' journal data in cloud storage	
	Sprint 3	AI GPT bot (configured for mental health awareness like Replika)	Crisis Management/ Chatbot
Beta Release	Sprint 4	Implement tools for users to set personal mental health goals and track their progress.	Goal Setting and Progress Tracking
		Provide motivational content and reminders to help users stay on track.	
	Sprint 5	Curate a library of guided meditations specifically designed to alleviate symptoms of depression and anxiety.	Guided Meditations

18th Nov 2024		AI to recommend meditations and articles based on mood.	
	Sprint 6	Create a community forum	Community Support
	Sprint 7	Develop a secure messaging feature for users to communicate with their therapists.	Secure Support
		AI to recommend meditations and articles based on mood	Mental Health Education
	Sprint 8	Create a journaling feature that provides AI-generated open-ended prompts to encourage reflective writing.	Personalised Content
		Ensure the AI is adaptive and responsive to user inputs, offering relevant and supportive prompts.	
	Sprint 9	Create options for security and features in settings	Customisation
		Create themes and layout options in settings.	
	Sprint 10	Create reminder tool	Reminders and Notifications
		Create notification tool	

Each sprint is performed with a rigorous structure, as shown below. This ensures that each sprint is effectively managed and that all critical aspects of development and testing are addressed promptly. Here's Sprint 1, for example -

Sprint 1 Goal: Finalise Mood Tracking Features

Each Sprint Duration: 4 weeks (28 days)

Sprint Timeline:

Day 1-10: Start initial coding

- Planning workshop

- Beginning sketching out front-end and back-end coding.
- Ensure all features meet the predefined acceptance criteria.

Day 11-15: Testing and Debugging

- Conduct extensive unit testing, integration testing, and system testing.
- Identify and fix critical bugs to ensure stability.
- Perform user acceptance testing (UAT) with a small group of target users to gather initial feedback.

Day 16-18: Quality Assurance and Refinement

- Conduct a thorough quality assurance (QA) review, focusing on usability, performance, and security.
- Refine UI/UX based on feedback from UAT and QA findings.
- Implement any minor enhancements that improve user experience without introducing new risks.

Day 19-20: Documentation and Preparation

- Finalise user documentation, including help guides and FAQs.
- Prepare release notes detailing the features, improvements, and known issues.
- Set up deployment pipelines and ensure all release processes are in place.

Day 21-24: Internal Testing Deployment

- Deploy the internally tested version of the app to a selected group of internal testers.
- Monitor the deployment process to address any issues that arise quickly.
- Ensure communication channels are open for immediate feedback from internal testers.

Day 25-28: Post-Testing Monitoring and Feedback Collection

- Monitor the app's performance and user feedback.
- Collect data on app usage, user satisfaction, and any reported issues.
- Begin planning for the next sprint based on the feedback and performance metrics.

A.1.c Contingencies

Considering the tasks we have allocated and defined, there are several contingencies that we may have to consider:

1. There may be resource constraints, especially with time, as we only have 9 weeks before our final project submission deadline; some sprints further along the timeline may be replanned where some app features will be prioritised for completion first so that we will at least have an app with minimum viable functionality to present during the final project submission.

2. Team members are unavailable due to other personal commitments. To be prepared for situations like this, we always ensure that at least two team members handle each task in case one has to take leave.

3. We may also encounter unforeseen technical issues during development, such as integration problems with APIs or unexpected bugs. As such, we will consult our lecturer and expert colleagues for help and promptly address unforeseen issues during development. Buffer time will be allocated to the project timeline for technical issues.

4. If we are to do beta testing and If beta feedback takes longer than expected, we will plan for additional iterations.

A.2 Product Form and Vision

A.2.a *Name*

SAGE

The name "SAGE" was selected to evoke wisdom and guidance, aligning with the app's role as a trusted companion in navigating mental health challenges. Its design emphasises inclusivity, with a user-friendly interface tailored to accommodate diverse needs and backgrounds, ensuring everyone can benefit from its resources and support.

A.2.b *Acronym*

The acronym "SAGE" encapsulates the app's holistic approach to mental health support: Supportive, Accessible, Guiding, and Empowering. These principles were carefully chosen to reflect our commitment to providing empathetic support, ensuring accessibility for all users, guiding individuals towards mental well-being through personalised plans, and empowering users to take control of their mental health journey.

S - Supportive	**A -** Accessible	**G -** Guiding	**E -** Empowering
• Offers empathetic and non-judgmental support. • Provides resources for mental health issues.	• Available 24/7 for *immediate assistance.* • User-friendly interface suitable for all age groups. • Inclusive design accommodating	• Offers personalised mental health plans and suggestions. • Guides users through meditation, mindfulness, and	• Empower users to take control of their mental health. • Encourages self-awareness and self-care practices.

• Encourages users through difficult times with positive reinforcement.	diverse needs and backgrounds.	coping strategies. • Provides a structured path to achieve mental well-being goals.	• Provides tools and resources to help users build resilience and confidence.

A.2.c *Logo*

Our logo embodies two symbols: an owl representing wisdom and guidance in mental health, and an angel symbolising compassion and supportive care. Together, they reflect our app's mission to provide insightful guidance and empathetic support, guiding users on their journey to mental well-being and offering comfort during difficult times.

A.2.d *Colour Scheme*

Following user feedback, we reevaluated our initial colour scheme of beige and light purple with black font due to concerns about its perceived lack of warmth and relaxation. Responding to this feedback, we refined our design approach by creating a second wireframe using soothing colours like sea green and blue. These selections were deliberate to cultivate a tranquil environment, enhancing the app's appeal as a calming and supportive resource for mental well-being. These colours will be used for the prototype.

A.3 App Feature Descriptions

Feature	**Details**
1. Mood Tracking	Develop a user-friendly interface for logging daily moods.
	Implement visualisation tools for users to track mood trends over time.
	AI will analyse data from mood tracking for trends and make a “forecast” of mood.
2. Goal setting and Progress Tracking	Implement tools for users to set personal mental health goals and track their progress.
	Provide motivational content and reminders to help users stay on track
3. Journaling with AI Prompts	Create a journaling feature that provides AI-generated open-ended prompts to encourage reflective writing.
	Ensure the AI is adaptive and responsive to user inputs, offering relevant and supportive prompts.

	Data is also used to supplement the AI for mood trend analysis.
4. Guided Meditations and Useful Articles	Curate a library of guided meditations specifically designed to alleviate symptoms of depression and anxiety.
	Integrate audio and video content seamlessly within the app.
	AI to recommend meditations and articles based on mood trends.
5. Chatbot	AI GPT bot (configured for mental health awareness like Replika)
6. Secure Therapist Sessions	Develop a secure message feature for users to communicate with their therapists.
	Implement encryption and data protection measures to ensure privacy.
7. Community Forum	Moderated forums (by both bot and human) for users to mingle and share their journey of struggles and recoveries.

A.4 Survey and Interview Results

To gain insight into the stakeholders, we considered the largest stakeholders of the project and designed a set of interviews for each stakeholder and interviewed them. Our aim was to get insight from two potential end users, a mental health professional and a data protection officer, as we understand the highly sensitive nature of data that would be collected and value its privacy and security. The questions in Appendix A were asked during the interview and grouped by stakeholder (Tables A1, A2 and A3)

We also designed a survey and disseminated it to our connections for a wider audience reach. The questions and their related options contained in Appendix C were asked in the survey.

The following were the assumptions we made about the features of the app, and also the validation that users have given us through the surveys and interviews:

Assumptions	Validation
Users need a simple and intuitive way to track their moods daily	**Guided meditations** and **mood tracking** were consistently mentioned as valuable features.
Users prefer open-ended journaling prompts that are relevant to their mental health	**Common dislikes** included paywalls, repetitive content, generic responses from chatbots, and a lack of in-depth solutions.

Secure communication with therapists is crucial for user trust	**Therapist communication** was also considered important by multiple subjects. All subjects expressed **high concern about the privacy and security of their data** when using mental health apps.
The app's interface should be easy to navigate, even for users with low technical proficiency	A preference for **clear, simple**, and **minimalistic** interfaces, with easy navigation and minimal complexity highlighted as important features.
Users will engage more with personalised content and recommendations	While **personalised** content and recommendations were generally seen as useful, they should be **optional** and **non-intrusive**.While **personalised** content and recommendations were generally seen as useful, they should be **optional** and **non-intrusive**.
The app should be accessible to users with varying levels of visual and cognitive abilities	While specific accessibility needs were not highlighted by most interviewees, **ensuring readability** and **easy-to-use features** was mentioned.

A.5 UAT form

User Acceptance Testing (UAT) Form

Project Name: Mental Health Companion App

Version: 1.3

Date: [Insert Date]

Tested By: [Insert Name]

Test Environment:

1. **Operating System:** [insert version of OS being used (e.g., macOS Ventura 13.5, Windows)]
2. **Devices:** [insert The type of device(s) used for testing (e.g., MacBook Pro, Acer etc.)]
3. **Browser/Software Version:** [Insert the version of any web browser or software being used (e.g., Google chrome 116.0.5845.179)]
4. **Network Configuration:** [insert network details that might be relevant (e.g., Wi-Fi connection, VPN settings, local network etc)

5. **Backend/Frontend Environment:** Staging environment, with a test database and API endpoints.

---

1. General Information

- **Name of Tester:**
- **Role:**

[any of the below -

1. **End User:** A person who uses the app in a typical, non-technical manner.
2. **Tester/QA Engineer:** Someone responsible for systematically testing the software for bugs and usability issues.
3. **Project Manager:** A person overseeing the project, who may focus on ensuring the app meets the business requirements.
4. **Developer:** A team member involved in building the app, focusing on specific functionalities.
5. **Subject Matter Expert (SME)**: An expert in the mental health field, who might test the app for relevance and accuracy in content.
6. **Client Representative:** A person representing the client or stakeholders, ensuring the product meets their expectations.]

- **Contact Information:**

2. Features

Feature 1: Journal Logging

Test Scenario	Test Steps	Expected Outcome	Actual Outcome	Pass/Fail	Comments
1.1. Logging a Journal Entry	1. Navigate to the journal page. 2. Write an entry. 3. Save the entry.	The entry is saved and can be retrieved later.	[Insert Outcome]	[Pass/Fail]	[Insert Comments]
1.2. Viewing Past Entries	1. Navigate to the journal log.	The past journal entry for the	[Insert Outcome]	[Pass/Fail]	[Insert Comments]

	2. Select a past date.	selected date is displayed.			

Feature 2: Chatbot

Test Scenario	**Test Steps**	**Expected Outcome**	**Actual Outcome**	**Pass/Fail**	**Comments**
2.1. Starting a Conversation	1. Navigate to the chatbot page. 2. Initiate a chat.	The chatbot responds appropriately to the user's input.	[Insert Outcome]	[Pass/Fail]	[Insert Comments]
2.2. Chat History	1. Initiate multiple conversations. 2. Review the chat history.	The chat history is accurately saved and retrievable.	[Insert Outcome]	[Pass/Fail]	[Insert Comments]

Feature 3. Mood Tracking

Test Scenario	**Test Steps**	**Expected Outcome**	**Actual Outcome**	**Pass/Fail**	**Comments**

3.1. Recording Mood	1. Navigate to the mood tracker page. 2. Select mood by dragging on the mood bar. 3. Enter your mood description. 4. Save the mood.	The mood is saved and can be viewed in history.	[Insert Outcome]	[Pass/Fail]	[Insert Comments]
3.2. Viewing Mood History	1. Navigate to the mood tracker log. 2. View past moods.	The saved mood history is displayed accurately.	[Insert Outcome]	[Pass/Fail]	[Insert Comments]

3. UI Testing

General Aesthetics

Test Scenario	Test Steps	Expected Outcome	Actual Outcome	Pass/Fail	Comments
4.1. Evaluate overall design consistency	Explore various pages (Landing, Home, Journal, Mood Tracker, Chatbot,	The app should maintain a consistent color scheme, typography, and design style throughout.	[Insert Outcome]	[Pass/Fail]	[Insert Comments]

	Insights).				
4.2. Check the visibility and legibility of text.	- Review text across different features. - Check on different devices and screen sizes.	All text should be clearly legible with proper contrast and font size.	[Insert Outcome]	[Pass/Fail]	[Insert Comments]
4.3 Assess the visual appeal of icons and buttons.	- Navigate through the app. - Interact with buttons and icons.	Icons and buttons should be visually appealing and enhance the user experience.	[Insert Outcome]	[Pass/Fail]	[Insert Comments]

Responsiveness and Adaptability

Test Scenario	**Test Steps**	**Expected Outcome**	**Actual Outcome**	**Pass/Fail**	**Comments**
5.1. Verify layout adjustments in landscape mode.	Rotate the device to landscape mode.	Layouts should adjust properly in landscape mode, without content overflow or misalignment.	[Insert Outcome]	[Pass/Fail]	[Insert Comments]
5.2 Evaluate the behaviour of UI elements on interaction	Interact with buttons, sliders, and text boxes.	UI elements should respond smoothly to interactions with appropriate feedback (e.g.,	[Insert Outcome]	[Pass/Fail]	[Insert Comments]

		button press animations).			

4. UX Testing

Navigation and User Flow

Test Scenario	Test Steps	Expected Outcome	Actual Outcome	Pass/Fail	Comments
6.1. Assess ease of navigating between app features.	Navigate through all major features (Journal, Mood Tracker, Chatbot).	The navigation should be intuitive, allowing easy access to all app features without confusion.	[Insert Outcome]	[Pass/Fail]	[Insert Comments]
6.2 Test the intuitiveness of the onboarding process.	- Register account with email and pw - Login with email and pw	New users should find the onboarding process straightforward and informative.	[Insert Outcome]	[Pass/Fail]	[Insert Comments]
6.3 Evaluate the accessibility of frequently used features.	- Access frequently used features such as mood tracking and journaling.	Frequently used features should be easily accessible from the main menu or homepage.	[Insert Outcome]	[Pass/Fail]	[Insert Comments]

Performance and Efficiency

Test Scenario	Test Steps	Expected Outcome	Actual Outcome	Pass/Fail	Comments
7.1. Test loading times of different features.	Open various features such as Journal, Mood Tracker, and Chatbot.	Features should load within a reasonable time (preferably under 2 seconds).	[Insert Outcome]	[Pass/Fail]	[Insert Comments]

5. Overall Feedback

- **Was the application easy to use?**
  [insert feedback]
- **Were there any issues or bugs encountered during testing?**
  [insert feedback]
- **Do you have any suggestions for improvements?**
- [insert feedback]

Acceptance

- **Are the features ready for release based on your testing?**
  [Yes/No]
- **If not, what areas need to be addressed before release?**
  [insert feedback]

Tester's Signature:
Date:

A.6 Meeting Notes and Materials

23rd July 2024

LLM based autonomous agent component
- document with framework
Profile user profile
memory documentation, chat history
reasoning (llm) GPT 4.0 "thinking" "thoughts" what does the user want? Intent
couselling or casual convo
tools/action API calls functions (int ext)

User prompt
1) retrieve user pf
2) what is the past chat
3) reasoning/thinking
4) does it require tools?
User response
4) Trigger tool or action based on response

Read the framework to prepare the report
Write some use cases/storylines to add to report
Then implement it

Authenticate user

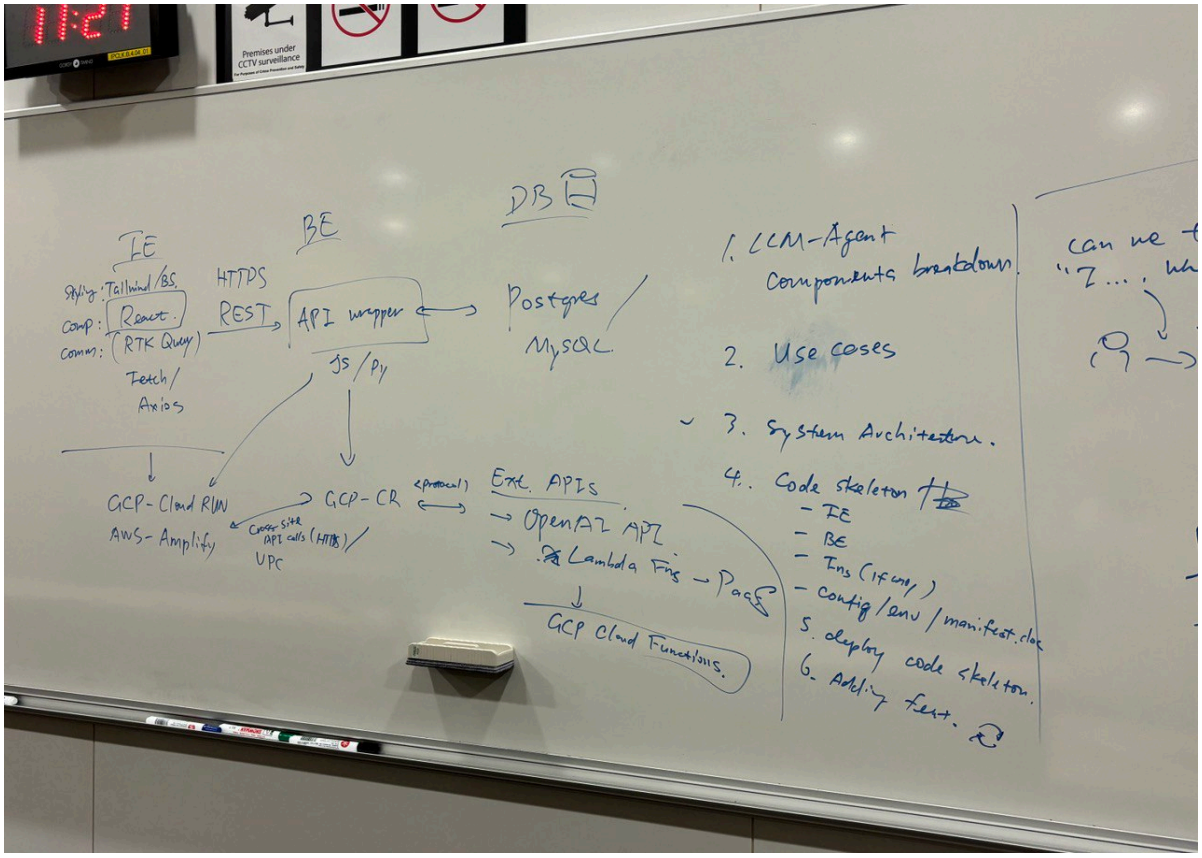
1) LLM agent
Component breakdown
2) Use cases
3) System Architecture
- JS/python (fast API?)
- draw out the full diagram on how each component will relate with each other
4) code skeleton
- IE BE Fns (if any) Config etc
- just have placeholder for now
get req ret hello world for example
- dummy connect everything
5) Deploy the code
6) Add features
- iterative

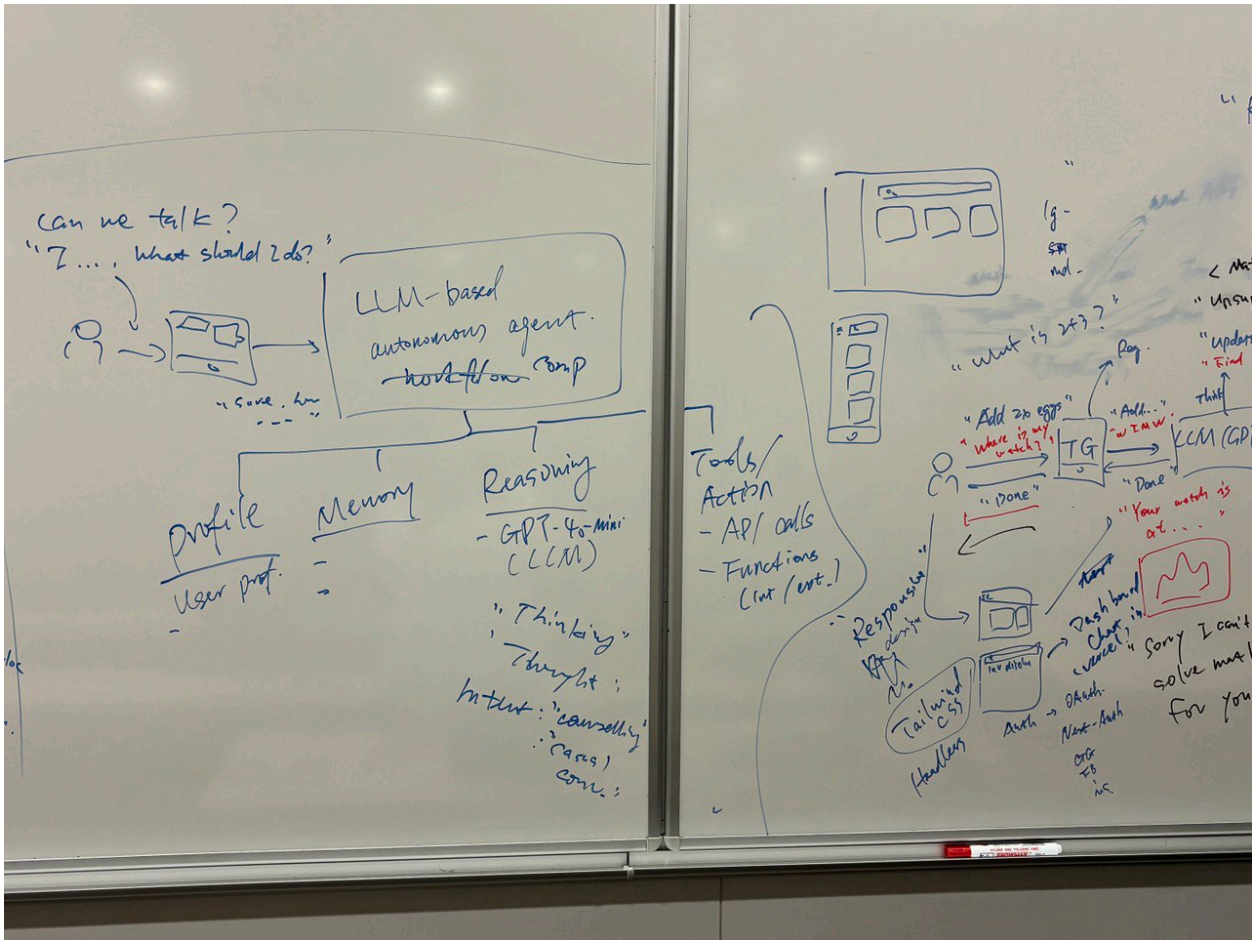
Lambda funct (mood analysis)

First chat case: casual conversation first.
Do it iteratively.

Deploy as sketelon
Then slowly deploy bitesized feature by feature (agile)

Target end of august to finish (6 weeks to finish everything)





27 july 2024

- ➔ Deadlines/Dates to note:
  - 27 July - meet to code
  - 10 August - Code Skeleton (complete)
  - 11-25 August - Workflow 1 (2 weeks)
  - 26-1 Sept - Workflow 2 (1 week)
    *End August - Final (complete everything)
  - after - any final bits we have to fix/code

General/add notes:

- ➔ Profile: can add on symptoms, past medical history
- ➔ Memory: journalling, chat history
- ➔ Reference whiteboard/article/notes for the LLM chart
  (to do: rewrite/tidy it, and add to proposal)
- ➔ Tailwind/BS, React, Axios, next.js node.js, will use JS (not PY), GPT 4.0 mini, AWS amplify
- ➔ (Lambda on hold for now)

- ➔ frontend: REACTJS stack
  backend: Next.js Node.js Stack
  database: SQlite
  third-party services: GPT3.5-turbo / GPT 4.0 mini
- ➔ Read the articles from lecturer, go through the tools (read up on documentation)
- ➔ Next meeting: Wed 31 July 5pm-8pm

<u>31 July 2024</u>

- ➔ 2 people frontend: mandy, lin
  3 people do backend: devon, sui rong, hui toon
- ➔ Do web app instead, responsive web app so it still works on mobile but online
- ➔ Use github
- ➔ Integrate on 7 August 2024
- ➔ Next meeting: Wed 7 August 5pm

11 References

Chant, T. (2023, June 21). *How to Build a ChatBot using the GPT-4 API – Full Project-Based Tutorial*. freeCodeCamp. Retrieved September 4, 2024, from https://www.freecodecamp.org/news/build-gpt-4-api-chatbot-turorial/#the-app-we-re-building

Mishra, A. (2024, March 7). *A Complete Guide To Serverless On AWS With Lambda*. Medium. https://medium.com/p/8b4aa04312fd

Starks, A. (2024, July 19). *GPT-4o mini vs GPT-3.5 Turbo. I just tried out the new model and am BEYOND Impressed*. Medium. https://medium.com/@austin-starks/gpt-4-mini-vs-gpt-3-5-turbo-i-just-tried-out-the-new-model-and-am-beyond-impressed-e7a6abb54b1f

Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z., & Wen, J. (2024, March 22). A survey on large language model based autonomous agents. *Springer Link*, *18*(186345), 26. https://doi.org/10.1007/s11704-024-40231-1

Roberts, M. (2017). Serverless Architectures on AWS: With Examples Using AWS Lambda. Manning Publications. https://www.manning.com/books/serverless-architectures-on-aws

Kleppmann, M. (2017). Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. O'Reilly Media. https://www.oreilly.com/library/view/designing-data-intensive-applications/9781491903063/

Luxton, D. D., et al. (2016). mHealth for Mental Health: Integrating Smartphone Technology in Behavioral Healthcare. Professional Psychology: Research and Practice. https://psycnet.apa.org/doi/10.1037/pro0000121

Bardus, M., et al. (2016). Mobile Health Apps for Weight Management: A Content Analysis. JMIR mHealth and uHealth. https://mhealth.jmir.org/2016/3/e87/